



UNIVERSIDADE PAULISTA
CIÊNCIAS DA COMPUTAÇÃO
76B9 - ATIVIDADES PRATICAS SUPERVISIONADAS (APS)

INTEGRANTES

ERIC TAKASHI KATAYAMA	RA: G7553I7
JOSÉ EWERTON ALVES DOS SANTOS	RA: G70JCI9
MARIANA KOBORI GABRIELLI	RA: R192512
GABRIEL HAMAD PASSOS ANDRADE	RA: G73GHC3

Tema: DESENVOLVIMENTO DE UM SISTEMA PARA ANÁLISE DE PERFORMANCE
DE ALGORITMOS DE ORDENAÇÃO DE DADOS

SÃO PAULO
2025

SUMÁRIO

1	Objetivo do Trabalho.....	4
2	Introdução.....	6
3	Fundamentação Teórica	9
3.1	Algoritmos de Ordenação.....	9
3.2	Conceito Geral de Ordenação.....	10
3.3	Bubble Sort.....	11
3.3.1	Funcionamento passo a passo.....	11
3.3.2	Características principais do Bubble Sort.....	12
3.3.3	Vantagens e desvantagens	12
3.4	Quick Sort.....	12
3.4.1	Exemplo prático.....	13
3.4.2	Características principais do Quick Sort.....	13
3.4.3	Vantagens e desvantagens	13
3.4.4	Exemplo de código (Python)	13
3.5	Insertion Sort	14
3.6	Selection Sort	14
3.7	Merge Sort.....	15
3.8	Heap Sort	15
4	Metodologia	16
4.1	Desenvolvimento do sistema.....	16
4.2	Estrutura de dados e leitura do arquivo.....	16
4.3	Implementação dos algoritmos de ordenação	18
4.4	Contagem e análise de desempenho	19
4.5	Testes e verificação do sistema.....	20
4.6	Considerações sobre o método.....	22

5	Projeto do programa	24
5.1	Arquitetura da Solução	24
5.1.1	Estrutura do arquivo	24
5.1.2	Estruturação das variáveis	24
5.1.3	Método de carregamento do arquivo.....	25
5.1.4	Funções comparadores.....	27
5.1.5	Ordenação por Bubble Sort.....	28
5.1.6	Ordenação por Quick Sort.....	29
5.1.7	Ordenação por Heap Sort	32
5.1.8	Exibição dos dados	35
5.1.9	Programa principal	36
5.2	Testes do programa.....	39
5.2.1	Funcionamento do programa	39
5.2.2	Validações e iterações do Bubble Sort.....	40
5.2.3	Validação e iteração do Quick Sort	41
5.2.4	Validação e iteração por Heap Sort.....	42
6	Discussão e Conclusão	44
6.1	Resumo dos dados de iteração e trocas	46
6.2	Bubble Sort.....	47
6.3	Quick Sort.....	48
6.4	Heap Sort	48
6.5	Conclusão.....	49
7	Relatório com as Linhas de Código do Programa	50
8	Bibliografia.....	60
9	Fichas de Atividades Práticas Supervisionadas	61

ÍNDICE DE FIGURAS

Figura 1 – Dados organizados no arquivo do Excel.	24
Figura 2 – Definição de constante e estruturação dos dados.....	25
Figura 3 – Método responsável pela abertura do arquivo e estruturação dos dados	26
Figura 4 – Métodos responsáveis por comparar dois valores de algum atributos. ...	28
Figura 5 – Código do método de ordenação Bubble Sort.....	29
Figura 6 – Método particionar: primeira parte do código da ordenação por Quick Sort	30
Figura 7 – Método quick_sort_recursivo: segunda parte do código da ordenação por Quick Sort.....	31
Figura 8 - Método quick_sort: terceira parte do código da ordenação por Quick Sort	32
Figura 9 - Método heapify: primeira parte do código da ordenação por Heap Sort ..	33
Figura 10 - Método heap_sort: segunda parte do código da ordenação por Heap Sort	34
Figura 11 – Código responsável em apresentar os resultados no console.....	35
Figura 12 – Primeira parte do código main: responsável por reservar espaço na memória para o vetor e abrir os arquivos csv.	36
Figura 13 – Segunda parte do código main: responsável por apresentar no console o menu de opções.	37
Figura 14 – Terceira parte do código main: responsável pela tomada de decisão e da apresentação de resultados no console.	38
Figura 15 – Finalização do código main.	39
Figura 16 – Impressão no console quando o arquivo não é encontrado.	39
Figura 17 – Impressão apresentado no console do menu de opções e a escolha da opção zero.....	40
Figura 18 – Resultado do console ao se digitar uma opção inválida.....	40
Figura 19 – Ordenação por Bioma pelo método Bubble Sort ordenado por bioma. .	41
Figura 20 – Últimas linhas da ordenação por Bubble Sort odenado por bioma.....	41
Figura 21 – Método de ordenação por Quick Sort ordenado por município mostrando as iterações e trocas realizadas.	42
Figura 22 – Método de ordenação por Heap Sort ordenado por data e mostrando as iteraões e trocas realizadas.....	43

1 OBJETIVO DO TRABALHO

O objetivo deste trabalho é desenvolver um sistema capaz de analisar a performance de diferentes algoritmos de ordenação de dados, aplicando-os às bases de dados reais sobre focos de incêndio detectados por satélite no estado de Alagoas. O sistema deverá permitir a organização eficiente dos dados com base em critérios como data, bioma, município e precipitação, além de apresentar o número de operações realizadas (comparações e trocas) por cada algoritmo utilizado. Com isso, busca-se compreender como a escolha adequada de algoritmos de ordenação pode influenciar a eficiência e o desempenho de aplicações que tratam grandes volumes de dados, especialmente em contextos ambientais que exigem respostas rápidas e precisas.

A motivação deste projeto surge da crescente necessidade de soluções tecnológicas que auxiliem o monitoramento ambiental e a prevenção de desastres naturais. Em Alagoas, os focos de incêndio representam um problema que, embora em menor escala comparado a outras regiões, impacta diretamente a vegetação nativa, a agricultura e o equilíbrio climático local.

A integração entre Computação e Meio Ambiente se mostra essencial para promover análises mais rápidas e confiáveis, que possam apoiar políticas públicas e estratégias de conservação.

Assim, ao unir os conhecimentos de Estrutura de Dados e algoritmos de ordenação com a aplicação prática em dados ambientais reais, este trabalho reforça a importância da Ciência da Computação como ferramenta de transformação social e de inovação sustentável.

2 INTRODUÇÃO

O avanço tecnológico das últimas décadas tem transformado profundamente a forma como os dados são coletados, armazenados e analisados. Com o crescimento exponencial da quantidade de informações disponíveis, impulsionado pelo uso de sensores, satélites, redes e dispositivos conectados, tornou-se indispensável o desenvolvimento de sistemas computacionais eficientes capazes de processar grandes volumes de dados com rapidez e precisão. Dentro desse contexto, a Ciência da Computação exerce um papel central, fornecendo métodos e ferramentas que viabilizam a extração de conhecimento a partir de dados brutos — processo essencial em diversas áreas, incluindo a ambiental.

No cenário atual, marcado por intensas transformações climáticas e pelo aumento de desastres naturais, as tecnologias de geoprocessamento e sensoriamento remoto ganharam destaque como instrumentos fundamentais no monitoramento de fenômenos ambientais. Entre esses fenômenos, os incêndios florestais se destacam por seu potencial destrutivo e pelo impacto direto na biodiversidade, no solo e na qualidade do ar. O Brasil, por abrigar uma vasta diversidade de biomas e uma grande extensão territorial, enfrenta um desafio constante no controle e prevenção de queimadas. Mesmo em estados de menor área, como Alagoas, a ocorrência de focos de incêndio representa uma ameaça significativa ao meio ambiente e à segurança das comunidades locais.

Os dados de satélite disponibilizados por instituições como o Instituto Nacional de Pesquisas Espaciais (INPE) permitem acompanhar, praticamente em tempo real, os focos de incêndio em todo o território nacional. Esses dados são fundamentais para compreender o comportamento das queimadas e direcionar ações de fiscalização e combate. Entretanto, devido ao grande volume e complexidade das informações coletadas, é necessário utilizar técnicas computacionais avançadas para organizar, processar e analisar esses dados de forma eficaz. É nesse ponto que o presente trabalho se insere, propondo o desenvolvimento de um sistema para análise de performance de algoritmos de ordenação de dados, aplicado às informações de focos de incêndio no estado de Alagoas.

A ordenação de dados é uma das operações mais básicas e importantes da computação. Ela permite organizar conjuntos de informações segundo critérios

específicos, facilitando a busca, o cruzamento e a interpretação dos resultados. Por trás dessa tarefa aparentemente simples, encontram-se diversos algoritmos com níveis variados de eficiência, consumo de memória e complexidade computacional.

Alguns exemplos clássicos são o Bubble Sort, Selection Sort, Insertion Sort, Merge Sort e Quick Sort, entre outros. Cada um deles possui características próprias que determinam o seu desempenho de acordo com o tamanho e o tipo de dados processados. Compreender o funcionamento e o comportamento desses algoritmos é essencial para escolher a solução mais adequada em cada contexto.

No contexto do geoprocessamento de dados ambientais, a eficiência do algoritmo de ordenação impacta diretamente o tempo de resposta e a qualidade das análises realizadas. Um sistema que processa milhões de registros de focos de incêndio precisa de métodos de ordenação otimizados, capazes de manipular grandes volumes de informação sem comprometer o desempenho. Por exemplo, ao ordenar os dados por data, é possível identificar períodos de maior incidência de queimadas; ao ordenar por bioma, torna-se viável observar quais ecossistemas estão mais vulneráveis; e ao ordenar por município ou precipitação, é possível relacionar a ocorrência de incêndios a fatores geográficos e climáticos específicos.

Dessa forma, a etapa de ordenação não apenas organiza os dados, mas também prepara o terreno para análises mais complexas, como a aplicação de técnicas de aprendizado de máquina (machine learning) e aprendizado profundo (deep learning). Essas tecnologias podem ser utilizadas, por exemplo, para classificar automaticamente áreas afetadas por incêndios ou para prever regiões de risco, com base em padrões históricos. Assim, a ordenação representa um passo inicial, porém crucial, em um processo analítico que integra conceitos de Estrutura de Dados, Inteligência Artificial e Ciência Ambiental.

O sistema proposto neste trabalho tem como propósito realizar a ordenação dos dados de focos de incêndio em Alagoas a partir de critérios predefinidos — como data, bioma, município e precipitação —, além de fornecer informações sobre o número de comparações e trocas realizadas durante o processo. A partir desses resultados, será possível avaliar e comparar o desempenho de diferentes algoritmos de ordenação, identificando qual apresenta melhor eficiência no tratamento de dados ambientais. Essa análise de performance permitirá compreender, de forma prática, a importância de conceitos estudados na disciplina de Estrutura de Dados e sua aplicação em situações reais.

Além do aspecto técnico, este trabalho também busca ressaltar a relevância social e ambiental da tecnologia. A integração entre computação e sustentabilidade é uma das tendências mais promissoras da atualidade, e o uso de ferramentas computacionais para a proteção dos ecossistemas representa uma contribuição concreta da Ciência da Computação para o desenvolvimento sustentável. Ao aplicar algoritmos a problemas ambientais reais, como o monitoramento de queimadas em Alagoas, os estudantes têm a oportunidade de unir teoria e prática, desenvolvendo não apenas habilidades técnicas, mas também uma visão crítica sobre o papel da tecnologia na sociedade.

Em síntese, esta introdução apresenta o cenário no qual o trabalho está inserido, contextualizando a importância da ordenação de dados e destacando a aplicabilidade dos algoritmos de Estrutura de Dados em situações do mundo real. O projeto visa, portanto, não apenas comparar o desempenho de diferentes métodos de ordenação, mas também demonstrar o potencial transformador da Computação na análise de dados ambientais. Espera-se que os resultados obtidos possam contribuir para o aprimoramento das técnicas de processamento e análise de grandes volumes de dados, fortalecendo o vínculo entre o aprendizado acadêmico e os desafios contemporâneos da sustentabilidade e da inovação tecnológica.

3 FUNDAMENTAÇÃO TEÓRICA

3.1 Algoritmos de Ordenação

Os algoritmos de ordenação são uma das funções fundamentais na área da ciência da computação. Eles são responsáveis por organizar determinado grupo de elementos, como números, palavras ou registros, de acordo com uma determinada sequência, mais usado em ordem crescente ou decrescente. Essa função, desempenha um papel essencial nos sistemas de computadores, pois muitas operações, como buscas, análises estatísticas e até mesmo o armazenamento em bancos de dados, dependem diretamente da forma como os dados estão ordenados.

A importância dos algoritmos de ordenação vai além da organização de listas. A escolha do método usado impacta diretamente no desempenho do programa, principalmente quando se tem um volume muito grande de informações. Em aplicações que exigem alta performance, a seleção de um algoritmo eficiente reduz o tempo de execução e o consumo de recursos do sistema, e no mundo real isso é o mais fundamental. Por isso, compreender os diferentes tipos de algoritmos de ordenação é fundamental tanto para o desenvolvimento de softwares quanto para o estudo da eficiência computacional.

De um modo geral, os algoritmos de ordenação são classificados em duas categorias principais, os comparativos e os não comparativos. Os algoritmos comparativos baseiam-se em comparações diretas entre pares de elementos, determinando suas posições na sequência ordenada. Já os não comparativos utilizam outras propriedades dos dados, como valores numéricos ou posições em um intervalo, para realizar a ordenação sem a necessidade de comparações diretas. Essa distinção é importante porque define não apenas a estratégia empregada pelo algoritmo, mas também o seu comportamento em relação à eficiência e à utilização de memória.

Além dessa classificação, cada algoritmo de ordenação pode ser usado sob diferentes perspectivas. Uma delas é a complexidade temporal, que mede o número de operações necessárias do programa para ordenar uma lista, e outra é a complexidade espacial, que indica a quantidade de memória adicional utilizada durante o processo. Outro aspecto relevante é a estabilidade, característica que indica se elementos com valores iguais mantêm sua ordem original após a ordenação.

Os algoritmos de ordenação também têm diferenças na forma de execução. Alguns são iterativos, realizam repetições sucessivas de operações até que a lista esteja completamente ordenada. Outros são recursivos, como o Quick Sort e o Merge Sort, que utilizam a estratégia conhecida como dividir para conquistar.

Esse recurso consiste em dividir a lista em partes menores, ordenar cada uma delas separadamente e, por fim, combinar os resultados em uma lista final totalmente organizada. A escolha entre um algoritmo iterativo ou recursivo depende do tipo de dado e da estrutura lógica utilizada para o problema.

Na prática, a eficiência e a escolha de um algoritmo de ordenação dependem diretamente da natureza dos dados e da razão em que é aplicado. Em sistemas que usa grandes quantidades de informações, como bancos de dados e aplicações científicas, a prioridade é a rapidez e o baixo consumo de recursos computacionais. Já em programas simples ou acadêmicos, são melhores algoritmos mais didáticos e de fácil compreensão, mesmo que sejam menos eficientes em termos de desempenho. Essa variedade de aplicações mostra que não existe um algoritmo de ordenação universalmente superior, depende do objetivo do uso necessário.

Estudar algoritmos de ordenação, é essencial para o desenvolvimento de análise de desempenho. Ele permite entender de forma mais aprofundada como as operações internas de um programa influenciam em sua velocidade e sua eficiência geral. Além disso, fornece uma base sólida para o aprendizado de outras técnicas avançadas de computação, como busca binária, árvores de dados e estruturas de indexação. Dominar esses algoritmos significa, compreender um dos fundamentos mais importantes da programação e da otimização de sistemas computacionais.

3.2 Conceito Geral de Ordenação

A ordenação é uma das tarefas mais usadas na programação, sendo muito importante sempre que se deseja organizar informações de maneira lógica e eficiente. Por exemplo, em listas de notas de alunos, nomes de clientes ou números de vendas, a ordenação permite que buscas sejam realizadas de forma mais rápida e que análises dos dados sejam mais precisas, pois já existem uma organização. Para atender a essa necessidade, foram desenvolvidos os algoritmos de ordenação, que têm como função reorganizar os elementos de uma lista de acordo com uma sequência pré-estabelecida, geralmente em ordem crescente ou decrescente.

O funcionamento de um algoritmo de ordenação baseia-se em comparações e trocas entre os dados, embora algumas abordagens mais avançadas utilizem métodos alternativos para reduzir o número de operações necessárias. A eficiência de um algoritmo, portanto, está relacionada tanto à quantidade de comparações e movimentações realizadas quanto à memória adicional que ele exige durante o processo. Além disso, diferentes algoritmos podem apresentar comportamentos distintos dependendo do tamanho da lista e da organização inicial dos dados, tornando importante a escolha do método mais adequado para cada situação.

Em termos práticos, compreender o conceito e o funcionamento dos algoritmos de ordenação é essencial para a programação, pois essa operação influencia diretamente o desempenho de sistemas e a qualidade das análises de dados. Apesar de ser uma tarefa aparentemente simples, a ordenação é um elemento central no desenvolvimento de soluções computacionais eficientes, servindo como base para a aplicação de técnicas mais complexas em diferentes áreas da computação.

<https://www.freecodecamp.org/portuguese/news/algoritmos-de-ordenacao-explicados-com-exemplos-em-python-java-e-c/>

3.3 Bubble Sort

O Bubble Sort é um dos algoritmos de ordenação mais simples e melhor para aprender, sendo ideal para compreender o processo de comparação e troca de valores. Seu nome, “bolha”, vem da forma como os maiores elementos “sobem” para o final da lista a cada passagem, de maneira semelhante as bolhas subindo à superfície da água. O algoritmo anda a lista várias vezes, comparando pares e elementos. Sempre que os elementos estiverem em ordem aleatória, suas posições são trocadas. Esse processo se repete até que nenhuma troca seja necessária, indicando que a lista está totalmente ordenada.

3.3.1 Funcionamento passo a passo

Suponha a seguinte lista de números:
[5, 3, 4, 1]

O algoritmo executa as seguintes etapas:

1. Primeira passada:

- a. Compara 5 e 3 → troca → [3, 5, 4, 1]

- b. Compara 5 e 4 → troca → [3, 4, 5, 1]
 - c. Compara 5 e 1 → troca → [3, 4, 1, 5]
- 2. Segunda passada:
 - a. Compara 3 e 4 → ok
 - b. Compara 4 e 1 → troca → [3, 1, 4, 5]
- 3. Terceira passada:
 - a. Compara 3 e 1 → troca → [1, 3, 4, 5]

A lista está ordenada.

Exemplo extraído do Google

https://www.w3schools.com/dsa/dsa_algo_bubblesort.php

3.3.2 Características principais do Bubble Sort

Estável

Tipo: Algoritmo de comparação, iterativo

In-place

3.3.3 Vantagens e desvantagens

Vantagens	Desvantagens
Fácil de entender e usar	Muito lento para listas grandes
Ideal no mundo acadêmico	Faz muitas comparações desnecessárias
Pode ser otimizado com pequenas modificações	Pouco utilizado no mundo corporativo.

3.4 Quick Sort

O Quick Sort é um algoritmo de ordenação muito eficiente e muito usado em tarefas do mundo real. Ele usa a técnica de dividir para conquistar, que consiste em dividir o problema em partes menores e resolver cada uma delas separadamente. O processo inicia-se com a escolha de um pivô, que é um elemento da lista. Em seguida, a lista é dividida em duas partes, uma tendo os elementos menores que o pivô e outra contendo os elementos maiores. O Quick Sort é então usado de forma recursiva a

essas duas partes, e, por fim, os resultados das sublistas ordenadas são combinados para formar a lista final ordenada.

3.4.1 Exemplo prático

Lista inicial: [8, 3, 1, 7, 0, 10, 2]

Escolha do pivô: 7

- Menores que 7 → [3, 1, 0, 2]
- Maiores que 7 → [8, 10]

Aplicando novamente o Quick Sort nas duas sublistas, obtemos:

- [3, 1, 0, 2] → [0, 1, 2, 3]
- [8, 10] → [8, 10]

Juntando tudo → [0, 1, 2, 3, 7, 8, 10]

Exemplo extraído do Goggle

<https://pt.khanacademy.org/computing/computer-science/algorithms/quick-sort/a/overview-of-quicksort>

3.4.2 Características principais do Quick Sort

- Tipo: Recursivo, comparativo
- Estável: Não necessariamente
- In-place:

3.4.3 Vantagens e desvantagens

Vantagens	Desvantagens
muito eficiente em listas grandes	Desempenho pode piorar com pivô mal escolhido
Implementação simples	Não é estável
Bastante usado em sistemas reais	Requer uso de recursão

3.4.4 Exemplo de código (Python)

```
def quick_sort(lista):
```

```

    if len(lista) <= 1:
        return lista
    else:
        pivo = lista[len(lista) // 2]
        menores = [x for x in lista if x < pivo]
        iguais = [x for x in lista if x == pivo]
        maiores = [x for x in lista if x > pivo]
        return quick_sort(menores) + iguais + quick_sort(maiores)
numeros = [8, 3, 1, 7, 0, 10, 2]
print("Ordenado:", quick_sort(numeros))

```

Exemplo extraído do Goggle

<https://www.freecodecamp.org/portuguese/news/algoritmos-de-ordenacao-explicados-com-exemplos-em-python-java-e-c/>

3.5 Insertion Sort

O Insertion Sort, é um algoritmo que ordena a lista de maneira gradual. Ele percorre elemento por elemento, inserindo cada novo valor na posição correta em relação aos elementos que já foram organizados. Esse método é simples e eficiente, principalmente quando aplicado a listas pequenas ou quase ordenadas. Em termos de complexidade, sua média é $O(n^2)$, mas no melhor caso, quando a lista já está quase ordenada, pode atingir $O(n)$. O Insertion Sort é um algoritmo estável e in-place, o que significa que não requer memória adicional significativa além da própria lista. Um exemplo prático que ilustra seu funcionamento é o modo como organizamos cartas nas mãos: a cada nova carta que recebemos, ela é inserida no lugar correto entre as cartas que já estão ordenadas.

3.6 Selection Sort

O Selection Sort, ou ordenação por seleção, apresenta um funcionamento distinto dos outros algoritmos. Ele percorre a lista em busca do menor elemento e o coloca na primeira posição. Em seguida, localiza o segundo menor elemento e o posiciona na segunda posição, repetindo esse processo até que toda a lista esteja ordenada. Apesar de sua simplicidade e facilidade de compreensão, o Selection Sort

realiza muitas comparações e poucas trocas, o que resulta em uma complexidade média de $O(n^2)$. Além disso, trata-se de um algoritmo não estável, o que significa que elementos iguais podem ter sua ordem relativa alterada após a ordenação. Devido a essas limitações, seu uso em aplicações práticas é reduzido, sendo mais comum em contextos didáticos ou para listas de pequeno tamanho.

3.7 Merge Sort

O Merge Sort, ou ordenação por intercalação, utiliza o mesmo princípio do Quick Sort, baseado na estratégia de dividir para conquistar. O algoritmo consiste em dividir a lista em duas partes, ordenar cada uma separadamente e, em seguida, mesclar as duas sublistas em uma única sequência ordenada. Sua principal vantagem está no desempenho consistente, apresentando complexidade $O(n \log n)$ tanto no melhor quanto no pior caso. O Merge Sort é um algoritmo estável, o que preserva a ordem relativa de elementos iguais, porém não é in-place, pois requer memória adicional para armazenar as sublistas durante o processo de intercalação. Devido a essas características, o Merge Sort é frequentemente utilizado em sistemas que demandam previsibilidade no tempo de execução, como aplicações críticas e bases de dados de grande porte.

3.8 Heap Sort

O Heap Sort, ou ordenação por heap, utiliza uma estrutura de dados conhecida como heap binário, que permite acessar de forma rápida o maior ou o menor elemento de um conjunto. O algoritmo inicia construindo um heap a partir da lista e, em seguida, remove repetidamente o maior elemento, inserindo-o no final da lista ordenada. Seu desempenho é consistente, apresentando complexidade $O(n \log n)$ em todos os casos. Trata-se de um algoritmo in-place, o que significa que não exige memória adicional significativa, mas não é estável, podendo alterar a ordem relativa de elementos iguais. Devido a essas características, o Heap Sort é especialmente indicado em situações em que há restrições de memória e é necessário garantir eficiência na ordenação.

https://www.cos.ufrj.br/~rfarias/cos121/aula_09.html

4 METODOLOGIA

4.1 Desenvolvimento do sistema

O desenvolvimento do sistema foi orientado pela proposta de criar uma aplicação capaz de ler, ordenar e exibir dados reais de focos de queimadas no estado de Alagoas. A metodologia aplicada baseou-se em uma abordagem prática, com o objetivo de consolidar o aprendizado sobre estruturas de dados e análise de algoritmos. O projeto foi implementado na linguagem C, escolhida por sua eficiência, controle de baixo nível e aplicabilidade no estudo de algoritmos clássicos. Segundo Kernighan e Ritchie (2014), o C é uma linguagem estruturada que possibilita compreender a lógica interna de manipulação de memória e o funcionamento detalhado das operações de ordenação, sendo ideal para fins didáticos e de desempenho.

O sistema foi desenvolvido no ambiente Code::Blocks, em plataforma Windows, utilizando a codificação UTF-8 para garantir a correta exibição de caracteres acentuados nos nomes de municípios e biomas. Para isso, foi utilizada a função `SetConsoleOutputCP(65001)`, conforme a documentação oficial da Microsoft (2023). A aplicação foi construída para operar em modo texto, priorizando a clareza e o desempenho. Essa escolha metodológica segue a recomendação de Wirth (1986), que destaca que a simplicidade estrutural e a clareza de propósito devem prevalecer sobre a complexidade visual, principalmente em sistemas voltados à análise de algoritmos.

A metodologia de implementação foi incremental (Pressman, 2016), ou seja, o sistema foi desenvolvido e testado em partes independentes até formar um conjunto coeso e funcional. Inicialmente, definiu-se a estrutura de dados principal e a leitura dos arquivos CSV. Em seguida, foram criadas as funções de comparação e, por último, as rotinas de ordenação, cada uma em sua respectiva função. Após a conclusão das implementações, iniciou-se o processo de testes e validação de desempenho.

4.2 Estrutura de dados e leitura do arquivo

Para armazenar os dados obtidos do Instituto Nacional de Pesquisas Espaciais (INPE), o programa define uma estrutura chamada `Foco`, que contém os seguintes

campos: `id_bdq`, `foco_id`, `lat`, `lon`, `data_pas`, `pais`, `estado`, `municipio` e `bioma`. Essa estrutura foi criada para representar cada registro de forma organizada e permitir o acesso direto aos elementos durante as operações de ordenação. Conforme Manzano (2017), o uso de estruturas compostas (structs) é essencial em projetos que lidam com dados heterogêneos, pois possibilita o agrupamento de diferentes tipos de variáveis em uma única entidade lógica.

O vetor de registros foi criado de forma dinâmica, utilizando a função `malloc()` para alocar memória suficiente para até mil elementos. Essa abordagem foi escolhida por sua flexibilidade, permitindo o carregamento de um número variável de registros sem desperdiçar recursos. A gestão dinâmica da memória é uma das principais vantagens da linguagem C, pois oferece controle direto ao programador (KERNIGHAN; RITCHIE, 2014). Assim, o sistema pode manipular eficientemente os dados obtidos nos arquivos CSV, sem limitações impostas por tamanhos fixos.

A leitura dos dados foi implementada na função `carregar_csv()`, que utiliza as funções padrão da biblioteca C, como `fopen()`, `fgets()` e `strtok()`. Essa função é responsável por abrir os arquivos correspondentes aos anos de 2023 e 2024, pular o cabeçalho e ler cada linha dos registros. A função `strtok()` é empregada para separar os campos delimitados por vírgulas, armazenando cada valor na posição correspondente dentro da estrutura `Foco`. Segundo Tanenbaum (2015), o processamento de arquivos texto estruturados é uma técnica eficiente para simular grandes volumes de dados, permitindo a validação de algoritmos de forma prática e reprodutível.

Ao término da leitura, os registros são armazenados sequencialmente em memória, e a variável `total` mantém o controle do número de focos lidos. Essa contagem é fundamental para determinar os limites de iteração nos algoritmos de ordenação. Para garantir a integridade das informações, a função realiza verificações de erro na abertura do arquivo, exibindo mensagens claras ao usuário em caso de falhas — prática recomendada por Pressman (2016) para o desenvolvimento de sistemas robustos e confiáveis.

4.3 Implementação dos algoritmos de ordenação

A etapa central da metodologia consistiu na implementação de três algoritmos clássicos de ordenação: Bubble Sort, Quick Sort e Heap Sort. A escolha desses métodos permitiu comparar abordagens distintas — uma simples e didática, outra recursiva e eficiente, e a terceira baseada em estrutura de heap. Essa comparação é de extrema importância no estudo de Estrutura de Dados, pois possibilita observar na prática o comportamento e a complexidade de cada algoritmo (CORMEN et al., 2012).

O Bubble Sort foi implementado por meio da função `bubble_sort()`. Seu funcionamento baseia-se na comparação sequencial de elementos adjacentes, realizando trocas sempre que o item atual é maior que o próximo. O processo se repete até que a lista esteja completamente ordenada. Esse método é caracterizado por sua simplicidade e alto valor didático, sendo amplamente utilizado em contextos de ensino. Entretanto, apresenta complexidade $O(n^2)$, o que o torna ineficiente para grandes volumes de dados (CORMEN et al., 2012). No sistema, essa função também contabiliza o número de iterações e trocas, fornecendo uma métrica precisa de desempenho.

O Quick Sort, implementado nas funções `quick_sort_recursivo()` e `particionar()`, utiliza a técnica de divisão e conquista (divide and conquer). Nesse método, o vetor é particionado em subvetores com base em um pivô, e a ordenação é aplicada recursivamente a cada parte. A eficiência do Quick Sort é evidenciada em bases de dados médias e grandes, apresentando complexidade média de $O(n \log n)$. De acordo com Knuth (1998), esse algoritmo é um dos mais eficientes entre os métodos de ordenação por comparação, principalmente quando bem implementado com controle de trocas e iterações.

Já o Heap Sort foi implementado nas funções `heapify()` e `heap_sort()`, baseando-se na estrutura de árvore binária. Nesse processo, o vetor é transformado em um heap, e o maior elemento é movido para o final do array a cada iteração. Esse método se destaca pela estabilidade e previsibilidade de desempenho, mantendo complexidade $O(n \log n)$ em todos os casos (SEdgeWICK; WAYNE, 2011). A função `heapify()` é chamada recursivamente para ajustar o heap a cada remoção de elemento, garantindo que a estrutura mantenha suas propriedades durante toda a execução.

Cada algoritmo foi implementado com parâmetros genéricos, permitindo que o usuário selecione diferentes critérios de ordenação. Para isso, o código define três funções comparadoras: `comparar_bioma()`, `comparar_municipio()` e `comparar_data()`. Cada uma delas utiliza a função `strcmp()` para comparar os campos de texto das estruturas, retornando valores positivos, negativos ou nulos conforme a ordem desejada. Essa abordagem modular, conforme recomendado por Pressman (2016), facilita a manutenção e a expansão do código, permitindo adicionar novos critérios de ordenação futuramente sem alterar a lógica principal do sistema.

4.4 Contagem e análise de desempenho

Um dos aspectos mais relevantes desta metodologia é a inclusão de contadores de desempenho nos algoritmos implementados. Cada método de ordenação registra o número de iterações (comparações realizadas) e trocas (substituições de elementos). Essa funcionalidade foi adicionada para tornar possível a análise quantitativa da eficiência de cada algoritmo. Segundo Cormen et al. (2012), a análise empírica de algoritmos é tão importante quanto a teórica, pois permite observar diferenças práticas em situações reais de processamento.

Durante a execução do Bubble Sort, os contadores são incrementados a cada comparação e a cada troca entre elementos adjacentes. Como a ordenação é realizada em duplo laço, o número de iterações cresce quadraticamente em função do tamanho do vetor, evidenciando sua baixa eficiência. Essa implementação possibilitou aos alunos visualizar de forma prática a diferença entre complexidade teórica e desempenho real. De acordo com Tenenbaum, Langsam e Augenstein (2009), a aprendizagem prática de estruturas de dados é consolidada quando o estudante observa empiricamente o comportamento dos algoritmos sob diferentes condições de entrada.

No caso do Quick Sort, as variáveis de contagem são passadas como ponteiros para as funções recursivas, garantindo que o número total de operações seja acumulado durante toda a execução. Isso é importante porque o Quick Sort divide o problema em várias chamadas recursivas e, sem esse controle, a contagem seria parcial. A estrutura implementada segue as boas práticas de recursividade descritas por Sedgewick e Wayne (2011), em que os parâmetros de controle são mantidos de

forma global para garantir a precisão dos resultados. O uso de recursão torna o Quick Sort eficiente e elegante, mas também exige um controle rigoroso do estado do programa, o que foi cuidadosamente tratado nesta metodologia.

O Heap Sort também faz uso de contadores de iterações e trocas. A função `heapify()` é chamada repetidamente para manter a estrutura de heap após cada extração de elemento. Esse processo permite verificar que, mesmo com várias comparações internas, o algoritmo mantém um comportamento estável e previsível. Segundo Knuth (1998), o Heap Sort é considerado um dos algoritmos mais consistentes em termos de complexidade, apresentando desempenho confiável independentemente da ordenação inicial dos dados. A implementação adotada neste trabalho reflete exatamente essa característica, demonstrando eficiência uniforme nos testes realizados.

A contagem de operações e a exibição dos resultados foram realizadas diretamente no console, após a execução de cada algoritmo. O programa apresenta ao usuário o número total de iterações e trocas, seguido dos dados ordenados. Essa abordagem interativa é fundamental em contextos educacionais, pois transforma o código em uma ferramenta de experimentação, em vez de apenas um produto funcional. Conforme Tanenbaum (2015), sistemas voltados ao aprendizado devem fornecer feedback direto ao usuário, permitindo a compreensão imediata do comportamento interno dos algoritmos.

4.5 Testes e verificação do sistema

Após a implementação dos algoritmos e dos contadores, foi realizada a etapa de testes e validação. Essa fase teve como propósito garantir que o sistema cumprisse corretamente todas as suas funções — desde a leitura e armazenamento dos dados até a ordenação e exibição dos resultados. Segundo Sommerville (2011), o processo de verificação de software é essencial para assegurar a confiabilidade do produto e sua aderência aos requisitos definidos.

Os testes iniciaram com a verificação da função de leitura dos arquivos CSV. Foram carregados os arquivos referentes aos anos de 2023 e 2024, ambos contendo dados reais de focos de queimadas no estado de Alagoas. O programa foi capaz de processar corretamente os arquivos e armazenar os registros em memória sem perda

de informações. A exibição dos dados no console confirmou a integridade da estrutura Foco, garantindo que todos os campos (ID, data, coordenadas, bioma, município e estado) estavam sendo preenchidos corretamente.

Na etapa seguinte, foram testadas todas as opções do menu interativo. O sistema apresenta nove possibilidades de execução, que combinam os três algoritmos de ordenação (Bubble, Quick e Heap Sort) com os três critérios de ordenação disponíveis (bioma, município e data). Cada opção foi testada individualmente, observando-se o comportamento do programa, a precisão da ordenação e a correspondência dos registros. Em todas as execuções, os resultados apresentados foram coerentes com o esperado, confirmando a funcionalidade das comparações e a correção da ordenação.

Durante os testes, também foi validada a robustez do sistema em situações de erro. Quando o arquivo CSV não era encontrado, o programa exibia mensagens de erro claras, conforme implementado na função `carregar_csv()`. Essa prática de tratamento de exceções está alinhada com as recomendações de Pressman (2016), que reforça a importância de fornecer respostas compreensíveis ao usuário, evitando falhas silenciosas ou comportamentos inesperados.

Outra etapa importante foi o teste de desempenho prático. Embora o foco do trabalho não seja medir tempos de execução com precisão, foi possível observar diferenças significativas entre os algoritmos a partir do número de iterações e trocas. O Bubble Sort apresentou os maiores valores de contagem, o Quick Sort teve desempenho intermediário e o Heap Sort mostrou-se o mais eficiente, com o menor número de trocas. Esses resultados estão em conformidade com a literatura clássica (CORMEN et al., 2012; KNUTH, 1998), validando empiricamente as análises teóricas estudadas na disciplina.

Para garantir a consistência das saídas, o programa foi executado várias vezes com os mesmos arquivos de entrada. Em todas as execuções, a ordenação foi idêntica, o que demonstra a determinismo dos algoritmos utilizados e a confiabilidade da implementação. A função `exibir_focos()` também foi validada, garantindo a correta formatação e apresentação dos dados. As colunas de ID, data, latitude, longitude, bioma, município e estado foram ajustadas com espaçamento uniforme, facilitando a leitura e a comparação dos registros.

O conjunto de testes demonstrou que o sistema é funcional, estável e reproduzível, atendendo a todos os requisitos propostos na atividade. A metodologia de validação aplicada permitiu confirmar tanto o comportamento correto do programa quanto a eficácia dos algoritmos implementados. Esse processo reforça a importância da experimentação prática no ensino de estruturas de dados, conforme destacado por Tenenbaum, Langsam e Augenstein (2009), que consideram a prática laboratorial um componente essencial na formação de profissionais de computação.

4.6 Considerações sobre o método

A metodologia adotada neste trabalho destacou-se pela clareza e objetividade no desenvolvimento das etapas, permitindo a compreensão detalhada do funcionamento interno dos algoritmos e da estrutura de dados utilizada. O uso de uma linguagem de baixo nível como o C proporcionou o controle total sobre a memória e as operações, permitindo observar de forma concreta como as comparações e trocas afetam o desempenho dos métodos de ordenação. Essa relação direta entre código e resultado é um dos principais diferenciais pedagógicos do projeto.

A organização modular do código, com funções separadas para leitura, comparação, ordenação e exibição, garantiu manutenibilidade e clareza, qualidades essenciais em sistemas bem estruturados. Segundo Pressman (2016), a modularização é uma prática recomendada para aumentar a coesão e reduzir o acoplamento entre as partes do sistema, facilitando a depuração e futuras melhorias. No caso deste projeto, cada módulo cumpre uma função específica e independente, o que demonstra a aplicação correta desses conceitos.

Além do aspecto técnico, o trabalho também teve relevância didática. O sistema desenvolvido permitiu aos estudantes exercitar a lógica de programação, compreender o impacto da complexidade algorítmica e aplicar na prática conceitos fundamentais da disciplina de Estrutura de Dados. Como observam Cormen et al. (2012), a compreensão profunda de algoritmos só é alcançada quando teoria e prática se complementam em experimentos concretos. Dessa forma, a metodologia empregada não apenas atendeu aos requisitos acadêmicos, mas também promoveu uma vivência realista e formativa.

Portanto, a metodologia aplicada neste trabalho representou um processo completo de planejamento, implementação, validação e análise. Cada etapa foi cuidadosamente executada, resultando em um sistema funcional, eficiente e coerente com os objetivos da APS. A clareza da estrutura, a precisão dos algoritmos e a fidelidade dos resultados comprovam a adequação da metodologia utilizada, consolidando-a como um exemplo prático do uso de técnicas de programação estruturada e análise de performance em Ciência da Computação.

5 PROJETO DO PROGRAMA

5.1 Arquitetura da Solução

A arquitetura da solução foi desenvolvida com base na leitura e processamento dos dados provenientes dos arquivos CSV, de forma a estruturar e manipular informações inicialmente dispersas. Para isso, o programa foi organizado em funções específicas responsáveis por carregar os dados, estruturar os registros em memória e aplicar algoritmos de ordenação conforme o critério selecionado pelo usuário. Nos itens a seguir, são apresentadas e descritas as funções que compõem o sistema, bem como o papel de cada uma no processo de organização e exibição dos dados.

5.1.1 Estrutura do arquivo

Para solucionar o problema, foi necessário, primeiramente analisar a disposição dos dados. A Figura 1 ilustra como os dados estão organizados no arquivo Excel: todas as variáveis estão na primeira linha, enquanto seus respectivos valores encontram-se nas linhas subsequentes. Além disso, apenas uma coluna foi utilizada, com os valores separados por vírgulas. Compreendendo a forma de organização dos dados e identificando as variáveis de interesse, tornou-se possível iniciar a estruturação das variáveis no programa.

Figura 1 – Dados organizados no arquivo do Excel.

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
1	id_bdq,foco_id,lat,lon,data_pas,pais,estado,municipio,bioma														
2	1615355323,33cc0ead-041c-39e9-8489-cf78304e062c,	-9.206570	, -37.594180	,2023-01-05 16:05:00	,Brasil,ALAGOAS,CANAPI,Coatinga										
3	1615355326,00bc6a92-28b4-3b24-88b1-cc5ad8ebfcf8,	-9.024120	, -35.301450	,2023-01-05 16:05:00	,Brasil,ALAGOAS,MARAGOGI,Mata Atlântica										
4	1615729756,4eb99a62-a2c4-36d5-b7ed-cdfe3f65844,	-9.930060	, -36.286800	,2023-01-08 16:34:00	,Brasil,ALAGOAS,TEOTÃ" NIO VILELA,Mata Atlântica										
5	1615729922,4d4bdb25-1462-3dc4-828a-51c439aa567b,	-9.918810	, -36.286800	,2023-01-08 16:34:00	,Brasil,ALAGOAS,TEOTÃ" NIO VILELA,Mata Atlântica										
6	1615832020,7331caf8-424f-3a27-b7ad-639bca723b2c,	-8.925630	, -35.883350	,2023-01-09 15:39:00	,Brasil,ALAGOAS,IBATEGUARA,Mata Atlântica										
7	1615832021,33afb5d-4417-3b05-a29e-d3f1aefb1e98,	-8.928570	, -35.890110	,2023-01-09 15:39:00	,Brasil,ALAGOAS,IBATEGUARA,Mata Atlântica										
8	1615832022,5fce91ac-a1ff-31ab-9337-d59a23df75a1,	-8.929110	, -35.912290	,2023-01-09 15:39:00	,Brasil,ALAGOAS,IBATEGUARA,Mata Atlântica										
9	1616984589,3ca60468-5edf-31d0-95b6-d82b37d5850b,	-9.821370	, -36.430030	,2023-01-19 16:09:00	,Brasil,ALAGOAS,JUNQUEIRO,Mata Atlântica										

5.1.2 Estruturação das variáveis

A Figura 2 apresenta a estrutura utilizada para armazenar os dados lidos do arquivo. Inicialmente, foram definidas duas constantes: MAX_REGISTROS (linha 6), com valor igual a 1000, representando o número máximo de registros, e MAX_STRING (linha 7), com valor igual a 100, que estabelece o tamanho máximo das strings utilizadas.

Em seguida, foram declaradas as variáveis da estrutura Foco (linhas 9 a 19). O campo `id_bdq` foi definido como `int`, por se tratar de um identificador numérico. As variáveis `lat` e `lon` foram declaradas como `float`, pois apresentam valores de coordenadas geográficas. Já as variáveis `foco_id`, `data_pas`, `pais`, `estado`, `município` e `bioma` foram definidas como arrays de `char`, com tamanho máximo estabelecido por `MAX_STRING`, pois armazenam dados textuais.

Após a definição da estrutura e com o conhecimento da organização dos dados na planilha Excel, foi possível implementar uma função para realizar a leitura do arquivo e alocar corretamente cada valor nos campos correspondentes.

Figura 2 – Definição de constante e estruturação dos dados.

```

6  #define MAX_REGISTROS 1000
7  #define MAX_STRING 100
8
9  typedef struct {
10     int id_bdq;
11     char foco_id[MAX_STRING];
12     float lat;
13     float lon;
14     char data_pas[MAX_STRING];
15     char pais[MAX_STRING];
16     char estado[MAX_STRING];
17     char municipio[MAX_STRING];
18     char bioma[MAX_STRING];
19 } Foco;

```

5.1.3 Método de carregamento do arquivo

O método responsável pela leitura dos o arquivo CSV encontra-se ilustrado na Figura 3 (linhas 157 a 188). Esse procedimento, denominado `carregar_csv` recebeu três parâmetros: `filename`, contendo o nome do arquivo a ser processado; `focos`, ponteiro para o vetor de estruturas `Foco` no qual os dados foram armazenados; e `total`, ponteiro para um inteiro utilizado para registrar a quantidade de elementos já carregados, permitindo a continuidade da inserção em caso de leitura de múltiplos arquivos.

Inicialmente, o arquivo foi aberto no modo leitura (linha 158). Em seguida, realizou-se uma verificação para garantir que a abertura foi bem sucedida (linhas 159 a 161). Caso o arquivo não estivesse disponível (por exemplo, devido a corrompimento, localização incorreta ou inexistência), o programa exibiu uma mensagem de erro e encerrou a execução da função.

Posteriormente, foi declarado um vetor de caracteres com capacidade para 500 posições, utilizado para armazenar cada linha lida do arquivo (linha 162). A primeira linha foi descartada (linha 163), visto que correspondia ao cabeçalho contendo o nome das variáveis. Em seguida, uma variável inteira foi inicializada com o valor presente em total, representando a quantidade atual de registros carregados (linha 164).

Figura 3 – Método responsável pela abertura do arquivo e estruturação dos dados

```

157 int carregar_csv(const char *filename, Foco *focos, int *total) {
158     FILE *arquivo = fopen(filename, "r");
159     if (!arquivo) {
160         printf("Erro ao abrir arquivo: %s\n", filename);
161         return 0;
162     }
163     char linha[500];
164     fgets(linha, sizeof(linha), arquivo); // Pula cabeçalho
165     int count = *total;
166     while (fgets(linha, sizeof(linha), arquivo) && count < MAX_REGISTROS) {
167         linha[strcspn(linha, "\n")] = 0;
168         char *token = strtok(linha, ",");
169         if (token) focos[count].id_bdq = atoi(token);
170         token = strtok(NULL, ",");
171         if (token) strncpy(focos[count].foco_id, token, MAX_STRING - 1);
172         token = strtok(NULL, ",");
173         if (token) focos[count].lat = atof(token);
174         token = strtok(NULL, ",");
175         if (token) focos[count].lon = atof(token);
176         token = strtok(NULL, ",");
177         if (token) strncpy(focos[count].data_pas, token, MAX_STRING - 1);
178         token = strtok(NULL, ",");
179         if (token) strncpy(focos[count].pais, token, MAX_STRING - 1);
180         token = strtok(NULL, ",");
181         if (token) strncpy(focos[count].estado, token, MAX_STRING - 1);
182         token = strtok(NULL, ",");
183         if (token) strncpy(focos[count].municipio, token, MAX_STRING - 1);
184         token = strtok(NULL, ",");
185         if (token) strncpy(focos[count].bioma, token, MAX_STRING - 1);
186         count++;
187     }
188     fclose(arquivo);
189     *total = count;
190     return 1;
}

```

A leitura das linhas subsequentes foi realizada dentro de um laço while (linhas 165 a 185), que executou enquanto houvesse conteúdo para leitura e o limite máximo

estabelecido por MAX_REGISTROS não tivesse sido atingido. Para cada linha, o caractere de quebra de linha ao final foi removido, quando presente. Em seguida, utilizou-se a função strtok para segmentar a linha com base no delimitador vírgula, extraíndo sequencialmente cada campo do arquivo. Cada valor extraído foi então convertido para o tipo apropriado e atribuído aos respectivos campos da estrutura Foco, mediante verificações condicionais.

Ao término da leitura de cada registro, o contador count foi incrementado em uma unidade (linha 185). Após o processamento completo das linhas, o arquivo foi fechado (linha 186) e o valor atualizado do contador foi armazenado na variável apontada por total (linha 187). Por fim, a função retornou o valor 1 (linha 188), indicando que o procedimento de leitura foi concluído com sucesso.

5.1.4 Funções comparadores

Os métodos de comparação utilizados no programa encontram-se apresentados na Figura 4 (linhas 22 a 31). Esses processos tinham por finalidade comprar dois registros e retornar um valor inteiro que indicasse sua relação de ordem, possibilitando a organização dos dados em ordem crescente. Dessa forma, valores “alfabeticamente” menores eram posicionados antes dos maiores. Por exemplo, datas mais antigas foram consideradas menores que datas mais recentes, enquanto, no caso de strings, a letra “A” precedia a letra “Z”.

Como o objetivo consistiu em ordenar os registros com base nos campos bioma, município ou data, foram implementados três métodos específicos de comparação: comparar_bioma (linha 22), comparar_municipio (linha 26) e comparar_data (linha 30). Cada função recebeu dois ponteiros genéricos como parâmetros (const void *a e const void *b), conforme exigido pelo padrão de uso da função qsort.

Em cada método, os ponteiros foram convertidos para o tipo Foco permitindo o acesso ao campo correspondente por meio da notação ((Foco*)a)->campo (linhas 23, 27 e 31). A função strcmp() foi então empregada para realizar a comparação entre os valores dos campos, retornando um valor negativo caso o primeiro argumento precedesse o segundo, zero quando fossem equivalentes, e um valor positivo caso ocorresse o inverso.

O mesmo processo foi adotado nos três métodos, diferenciando-se apenas pelo atributo utilizado na comparação (bioma, município ou data).

Figura 4 – Métodos responsáveis por comparar dois valores de algum atributos.

```

22 int comparar_bioma(const void *a, const void *b) {
23     return strcmp(((Foco *)a)->bioma, ((Foco *)b)->bioma);
24 }
25
26 int comparar_municipio(const void *a, const void *b) {
27     return strcmp(((Foco *)a)->municipio, ((Foco *)b)->municipio);
28 }
29
30 int comparar_data(const void *a, const void *b) {
31     return strcmp(((Foco *)a)->data_pas, ((Foco *)b)->data_pas);
32 }

```

5.1.5 Ordenação por Bubble Sort

A Figura 5 apresentou a implementação do algoritmo Bubble Sort, responsável por ordenar o vetor de estruturas e contabilizar o número de iterações e trocas realizadas durante o processo. O método denominado `bubble_sort` (linha 34) recebeu três parâmetros: um ponteiro `focos`, que referenciava o vetor de estruturas `Foco`; o inteiro `total`, que indicava a quantidade de elementos a serem ordenados; e um ponteiro para o método `comparar`, utilizado para definir o critério de comparação entre dois registros.

Inicialmente, foram declaradas as variáveis `iterações` e `trocas`, ambas do tipo `long` e inicializadas com valor zero (linha 35). Essas variáveis tiveram como finalidade registrar, respectivamente, o número total de comparações realizadas e o número de vezes em que elementos trocaram de posição.

Na sequência, foram utilizados dois laços `for` (linhas 37 a 46) com a finalidade de percorrer todo o vetor. No laço interno, o método de comparação foi chamado para avaliar dois elementos adjacentes. Esse método retornava um valor negativo quando o primeiro elemento precedia o segundo, zero quando eram equivalentes e um valor positivo quando o primeiro deveria ser posicionado após o segundo (linha 40). Quando o retorno foi superior a zero, significando que os elementos estavam fora de ordem, seus valores foram trocados utilizando uma variável temporária do tipo `Foco`. Nessa

situação, a variável trocas foi incrementada. Adicionalmente, a cada comparação realizada, a variável iterações foi atualizada.

Ao término da execução, o método exibiu na tela o total de iterações e trocas efetuadas, permitindo assim avaliar o comportamento do algoritmo frente ao conjunto de dados processados.

Figura 5 – Código do método de ordenação Bubble Sort

```

34 void bubble_sort(Foco *focos, int total, int (*comparar)(const void *, const void *)) {
35     long iteracoes = 0, trocas = 0;
36
37     for (int i = 0; i < total - 1; i++) {
38         for (int j = 0; j < total - i - 1; j++) {
39             iteracoes++;
40             if (comparar(&focos[j], &focos[j + 1]) > 0) {
41                 Foco temp = focos[j];
42                 focos[j] = focos[j + 1];
43                 focos[j + 1] = temp;
44                 trocas++;
45             }
46         }
47     }
48
49     printf("\nIterações realizadas: %ld\n", iteracoes);
50     printf("Trocas realizadas: %ld\n", trocas);
51 }

```

5.1.6 Ordenação por Quick Sort

A ordenação por quick sort consistiu na escolha de um pivô e particionar o vetor em duas partes, nos menores que o pivô e nos maiores que o pivô. O código deste processo de ordenação foi dividido em três métodos: particionar, quick_sort_recursivo e quick_sort.

5.1.6.1 Método Particionar

O método particionar tem como objetivo selecionar um pivô e reorganizar o vetor de forma que todos os elementos menores que o pivô fiquem à sua esquerda e os maiores, à sua direita (linhas 55 a 79). Os parâmetros do método (linhas 55 a 57) são: o ponteiro *focos, do tipo Foco, que representa o vetor de elementos; os inteiros baixo e alto, que correspondem, respectivamente, aos índices inicial e final da parte do vetor a ser particionada; um ponteiro para função int (*comparar)(const void *, const void *), utilizada para comparar dois elementos; e, por fim, os ponteiros *iterações e

*trocas, que são contadores responsáveis por registrar o número de comparações e trocas realizadas durante a execução do algoritmo.

Nas linhas 59 e 60, o pivô é definido como o último elemento do intervalo (focos[alto]). Em seguida, declara-se a variável i, inicializada com (baixo - 1), que indica a posição do último elemento menor que o pivô.

A partição ocorre dentro de um laço for e uma estrutura condicional if (linhas 62 a 71). O laço percorre os elementos do vetor entre os índices baixo e (alto - 1). A cada iteração, o contador de iteração é incrementado. Dentro do if, a função comparar verifica se o elemento atual (focos[j]) é menor que o pivô. Caso seja, o índice i é incrementado e os elementos focos[i] e focos[j] são trocados de posição, registrando-se uma nova troca.

Figura 6 – Método particionar: primeira parte do código da ordenação por Quick Sort

```

55 int particionar(Foco *focos, int baixo, int alto,
56               int (*comparar)(const void *, const void *),
57               long *iteracoes, long *trocas) {
58
59     Foco pivo = focos[alto];
60     int i = baixo - 1;
61
62     for (int j = baixo; j < alto; j++) {
63         (*iteracoes)++;
64         if (comparar(&focos[j], &pivo) < 0) {
65             i++;
66             Foco temp = focos[i];
67             focos[i] = focos[j];
68             focos[j] = temp;
69             (*trocas)++;
70         }
71     }
72
73     Foco temp = focos[i + 1];
74     focos[i + 1] = focos[alto];
75     focos[alto] = temp;
76     (*trocas)++;
77
78     return i + 1;
79 }

```

Após o término do laço, o pivô é posicionado corretamente no vetor (linhas 73 a 76), sendo trocado com o primeiro elemento maior que ele (focos[i+1]). Dessa forma, os elementos menores ficam à esquerda e os maiores à direita, concluindo o processo de partição e retornando a posição final do pivô.

5.1.6.2 Método Quick sort recursivo

O método `quick_sort_recursivo`, representado pela Figura 7 (linhas 81 a 89), foi responsável pela divisão e ordenação do vetor. Seus parâmetros foram: o ponteiro `focos`, do tipo `Foco`, que representava o vetor de elementos; as variáveis `baixo` e `alto`, do tipo `int`, que delimitavam o intervalo atual da ordenação, sendo `baixo` o índice inicial e `alto` o índice final; o ponteiro para função `(*comparar)(const void *, const void *)`, utilizado para definir o critério de comparação (por município, data ou bioma); e por fim, os ponteiros `*iteracoes` e `*trocas`, que foram empregados para registrar o número de comparações e trocas realizadas, permitindo avaliar o desempenho do algoritmo.

O código iniciou-se com uma estrutura condicional (linha 84) que verificava se o valor de `baixo` era menor que o `alto`. Quando essa condição foi satisfeita, o método chamou a função `particionar` (linha 85), responsável por selecionar um pivô e reorganizar o vetor de modo que os elementos menores que o pivô ficassem à sua esquerda e os maiores, à sua direita, retornando a posição final do pivô.

Em seguida, o método foi chamado recursivamente duas vezes: a primeira (linha 86) para ordenar a subparte à esquerda do pivô – onde se encontravam os elementos menores – e a segunda (linha 87) para ordenar a subparte à direita – onde estavam os elementos maiores. Dessa forma, o algoritmo aplicou o princípio da divisão e conquista, repetindo o processo até que todas as partições estivessem ordenadas.

Figura 7 – Método `quick_sort_recursivo`: segunda parte do código da ordenação por Quick Sort.

```

81 void quick_sort_recursivo(Foco *focos, int baixo, int alto,
82                          int (*comparar)(const void *, const void *),
83                          long *iteracoes, long *trocas) {
84     if (baixo < alto) {
85         int pi = particionar(focos, baixo, alto, comparar, iteracoes, trocas);
86         quick_sort_recursivo(focos, baixo, pi - 1, comparar, iteracoes, trocas);
87         quick_sort_recursivo(focos, pi + 1, alto, comparar, iteracoes, trocas);
88     }
89 }

```

5.1.6.3 Método Quick Sort

Para a finalização do processo de ordenação, o último método implementado foi o `quick_sort` (Figura 8). O método em questão (linhas 91 a 96) foi responsável por inicializar o processo de ordenação, preparando as variáveis de controle para

contabilizar as iterações e as trocas, além de realizar a primeira chamada recursiva ao método `quick_sort_recursivo`, que realizou efetivamente a ordenação.

Os parâmetros do método foram: `focos`, do tipo `Foco`, um ponteiro para o vetor de elementos a ser ordenado; `total`, que indicava a quantidade total do vetor; e o ponteiro `(*comparar)(const void *, const void *)`, que chamava o método de comparação responsável por definir o critério de ordenação (por município, data ou bioma).

A primeira linha do método (linha 92) definiu as variáveis iterações e trocas, ambas inicializadas com o valor zero. Em seguida, o código chamou o método `quick_sort_recursivo` (linha 93) que realizou a divisão e organização do vetor a partir de um pivô. Por fim, o método exibiu o número de iterações (linha 94) e trocas (linha 95) realizadas, permitindo avaliar o desempenho do algoritmo.

Figura 8 - Método `quick_sort`: terceira parte do código da ordenação por Quick Sort

```

91 void quick_sort(Foco *focos, int total, int (*comparar)(const void *, const void *)) {
92     long iteracoes = 0, trocas = 0;
93     quick_sort_recursivo(focos, 0, total - 1, comparar, &iteracoes, &trocas);
94     printf("\nIterações realizadas: %ld\n", iteracoes);
95     printf("Trocas realizadas: %ld\n", trocas);
96 }

```

5.1.7 Ordenação por Heap Sort

O método de ordenação Heap Sort organizou os elementos do vetor em uma estrutura de heap binário e foi implementado em duas etapas principais: `heapify`, responsável por construir o heap, e `heap_sort`, que realizou a ordenação propriamente dita, removendo sucessivamente a raiz e reorganizando a estrutura para manter a propriedade de heap.

5.1.7.1 Método Heapify

Como já explicado, o método `heapify` (linhas 99 a 130) foi responsável por garantir a propriedade de heap em uma subárvore, assumindo que as subárvores já eram heap. A Figura 9 mostra que o código utilizado foi de heap binário máximo, o que significa que o valor do nó pai deveria ser igual ou maior que os valores de seus filhos.

Os parâmetros definidos foram: `focos`, do tipo `Foco`, que representou o heap; os inteiros `n` e `i`, sendo `n` utilizado para definir o tamanho do heap e `i` representando o índice do nó que estava sendo processado; o ponteiro `*comparar`, que chamava o

método de comparação para definir qual valor era maior ou menor; os ponteiros de iterações e trocas, que serviam como contadores para medir o desempenho.

A inicialização do código (linhas 103 a 105) definiu os inteiros maior, responsável por guardar o índice do maior valor entre raiz e filhos, e esquerda e direita, que calcularam os índices dos filhos de i em uma representação de vetor de heap binário. Em seguida, incrementou-se o contador de iterações em uma unidade (linha 107).

Figura 9 - Método heapify: primeira parte do código da ordenação por Heap Sort

```

99 void heapify(Foco *focos, int n, int i,
100             int (*comparar)(const void *, const void *),
101             long *iteracoes, long *trocas) {
102
103     int maior = i;        // Inicializa o maior como raiz
104     int esquerda = 2 * i + 1; // filho esquerdo
105     int direita = 2 * i + 2; // filho direito
106
107     (*iteracoes)++;
108
109     // Se o filho esquerdo é maior que a raiz
110     if (esquerda < n && comparar(&focos[esquerda], &focos[maior]) > 0) {
111         maior = esquerda;
112     }
113
114     // Se o filho direito é maior que o maior até agora
115     if (direita < n && comparar(&focos[direita], &focos[maior]) > 0) {
116         maior = direita;
117     }
118
119     // Se o maior não é a raiz
120     if (maior != i) {
121         // Troca a raiz com o maior elemento
122         Foco temp = focos[i];
123         focos[i] = focos[maior];
124         focos[maior] = temp;
125         (*trocas)++;
126
127         // Recursivamente faz heapify na sub-árvore afetada
128         heapify(focos, n, maior, comparar, iteracoes, trocas);
129     }
130 }

```

Após essas definições, aplicou-se um condicional if que verificava se o filho da esquerda existia e comparava com o de maior (linhas 110 a 112). Caso o filho esquerdo fosse maior, o índice maior foi atualizado. De forma similar, verificou-se o filho direito (linhas 115 a 117), atualizando maior se o valor do filho direito fosse maior.

Por fim, um terceiro condicional if (linhas 120 a 128) verificou se a raiz era o maior elemento. Quando o maior não era a raiz, realizou-se a troca entre a raiz e o maior elemento (linhas 121 a 123), incrementando-se o contador de trocas em uma

unidade. Após a troca, o próprio método `heapify` foi chamado recursivamente na subárvore afetada para garantir que a propriedade de heap fosse mantida.

5.1.7.2 Método Heap Sort

O método `heap_sort`, representado pelas linhas 132 a 154 na Figura 10, implementou a ordenação Heap Sort sobre o vetor de elementos. Ele utilizou o método `heapify` para construir o heap a partir do vetor original, e em seguida, extraiu os elementos em ordem, reorganizar o heap após cada extração. Ao final, o vetor tornou-se ordenado conforme o critério definido pelo método de comparação.

Os parâmetros do método foram: `focos`, do tipo `Foco`, um ponteiro para o vetor de elementos a ser ordenado; `total`, inteiro que indicou o número total de elementos do vetor; e o ponteiro `(*comparar)(const void*, const void*)`, que referiu ao método de comparação utilizado para definir a ordem entre os elementos.

A primeira linha do método (linha 133) definiu os contadores iterações e trocas, ambos inicializados com zero. Em seguida, procedeu-se à construção do heap por meio de um laço `for` (linhas 136 e 137) que percorreu os nós internos do heap do meio para o início do vetor (índices $\text{total}/2 - 1$ até 0). Para cada nó interno, o método `heapify` foi chamado, garantindo a propriedade de heap máximo; ao término desse laço, o vetor representou um heap binário válido.

Figura 10 - Método `heap_sort`: segunda parte do código da ordenação por Heap Sort

```

132 void heap_sort(Foco *focos, int total, int (*comparar)(const void *, const void *)) {
133     long iteracoes = 0, trocas = 0;
134
135     // Constrói o heap (reorganiza o array)
136     for (int i = total / 2 - 1; i >= 0; i--) {
137         heapify(focos, total, i, comparar, &iteracoes, &trocas);
138     }
139
140     // Extrai um elemento por vez do heap
141     for (int i = total - 1; i > 0; i--) {
142         // Move a raiz atual para o final
143         Foco temp = focos[0];
144         focos[0] = focos[i];
145         focos[i] = temp;
146         trocas++;
147
148         // Chama heapify no heap reduzido
149         heapify(focos, i, 0, comparar, &iteracoes, &trocas);
150     }
151
152     printf("\nIterações realizadas: %ld\n", iteracoes);
153     printf("Trocas realizadas: %ld\n", trocas);
154 }

```

Na etapa seguinte (linhas 141 a 150) realizou-se a ordenação propriamente dita: iterou-se do final do vetor para o início, movendo a raiz do heap (maior elemento) para a posição final `i` e decrementando-se o tamanho efetivo do heap. Após cada troca

entre a raiz e `focos[i]`, incrementou-se o contador de trocas e invocou-se novamente o método `heapify` sobre a raiz do heap reduzido, garantindo a manutenção da propriedade de heap máximo.

Por fim, o método exibiu o desempenho por meio dos contadores, imprimindo o número total de iterações e de trocas realizadas durante a execução.

5.1.8 Exibição dos dados

O método de `exibir_focos` Figura 11 foi responsável por definir como os dados armazenados no vetor do tipo `Foco` seriam exibidos na tela (linhas 196 a 211).

Seus parâmetros foram: o ponteiro `*focos`, do tipo `Foco`, que representou o vetor de elementos a ser exibido; o inteiro `total`, que indicou o número total de elementos do vetor e serviu para controle do laço de repetição.

As primeiras linhas do código determinaram como seria a formatação de cada campo e exibiram os títulos das colunas, como ID, Data, Bioma, Município e outros (linhas 197 e 198). A linha seguinte (linha 199) foi utilizada para imprimir um separador visual.

Em seguida, empregou-se um laço `for` (linhas 201 a 209) que percorreu todos os elementos do vetor. A cada iteração, foram impressos os valores dos campos da estrutura `Foco`, respeitando a formatação e a ordem de exibição definidas anteriormente.

Por fim, após a exibição de todos os registros, o método imprimiu o total de registros apresentados, confirmando a quantidade de dados processados.

Figura 11 – Código responsável em apresentar os resultados no console.

```

195 // ===== EXIBIÇÃO =====
196 void exibir_focos(Foco *focos, int total) {
197     printf("\n%-10s %-20s %-12s %-12s %-15s %-30s %-20s\n",
198         "ID", "Data", "Lat", "Lon", "Bioma", "Município", "Estado");
199     printf("-----\n");
200
201     for (int i = 0; i < total; i++) {
202         printf("%-10d %-20s %-12.6f %-12.6f %-15s %-30s %-20s\n",
203             focos[i].id_bdq,
204             focos[i].data_pas,
205             focos[i].lat,
206             focos[i].lon,
207             focos[i].bioma,
208             focos[i].municiplio,
209             focos[i].estado);
210     }
211     printf("\nTotal de registros: %d\n", total);
212 }

```

5.1.9 Programa principal

O programa main, detalhado entre as linhas 215 e 306 (Figura 12, Figura 13, Figura 14 e Figura 15) foi responsável por integrar todos os métodos desenvolvidos, gerenciar a execução do sistema e controlar o fluxo principal do programa.

A primeira parte do código (Figura 12) iniciou com a configuração da codificação de caracteres (linha 216), utilizando a função `SetConsoleOutputCP(65001)`. Essa configuração permitiu que caracteres especiais, como acentos e a letra “ç” fossem exibidos corretamente no console.

Em seguida, realizou-se a alocação dinâmica de memória por meio da função `malloc`, criando um vetor de estruturas do tipo `Foco`, com tamanho máximo definido pela constante `MAX_REGISTROS` (linha 217). O ponteiro `focos` foi utilizado para armazenar os registros carregados a partir dos arquivos de dados. Logo após, foi inserida uma condicional que verificava se a alocação de memória havia ocorrido corretamente (linha 218). Caso a alocação falhasse, era exibida uma mensagem de erro (linha 219) e o programa era encerrado.

Na sequência, definiu-se a variável `total` (linha 223), inicializada com o valor zero. Essa variável foi utilizada para armazenar o número total de registros carregados dos arquivos CSV.

Figura 12 – Primeira parte do código main: responsável por reservar espaço na memória para o vetor e abrir os arquivos csv.

```

215  int main() {
216      SetConsoleOutputCP(65001);
217      Foco *focos = malloc(MAX_REGISTROS * sizeof(Foco));
218      if (!focos) {
219          printf("Erro ao alocar memória!\n");
220          return 1;
221      }
222
223      int total = 0;
224
225      printf("Carregando dados...\n");
226      if (!carregar_csv("focos_br_al_ref_2023.csv", focos, &total)) {
227          free(focos);
228          return 1;
229      }
230      if (!carregar_csv("focos_br_al_ref_2024.csv", focos, &total)) {
231          free(focos);
232          return 1;
233      }

```

O passo seguinte consistiu na leitura dos arquivos contendo os dados, por meio do método `carregar_csv()`. Foram carregados dois arquivos, um referente ao ano de

2023 e outro ao de 2024, cujos dados foram armazenados no vetor focos (linhas 225 a 232). Cada chamada ao método foi encapsulada em uma estrutura condicional if que verificava o sucesso do carregamento. Caso ocorresse algum erro, como arquivo inexistente ou corrompido, a memória alocada era liberada e o programa encerrado. Ao final dessa etapa, o vetor focos continha os dados combinados de ambos os arquivos, e a variável total armazenava a quantidade total de registros carregados.

Para permitir que o usuário escolhesse o método de ordenação (Bubble Sort, Quick Sort ou Heap Sort) e o campo de comparação (Bioma, Município ou Data), foi implementado um menu de opções (linhas 235 a 249), conforme ilustrado na Figura 13. As opções de 1 a 9 representavam combinações entre método e campo de ordenação, enquanto a opção 0 permitia encerrar o programa. A leitura da escolha do usuário era realizado pela função scanf() (linha 249).

Figura 13 – Segunda parte do código main: responsável por apresentar no console o menu de opções.

```

235     int opcao;
236     do {
237         printf("\n=== SISTEMA DE ORDENAÇÃO DE FOCOS ===\n");
238         printf("1 - Ordenar por Bioma (Quick Sort)\n");
239         printf("2 - Ordenar por Município (Quick Sort)\n");
240         printf("3 - Ordenar por Data (Quick Sort)\n");
241         printf("4 - Ordenar por Bioma (Bubble Sort)\n");
242         printf("5 - Ordenar por Município (Bubble Sort)\n");
243         printf("6 - Ordenar por Data (Bubble Sort)\n");
244         printf("7 - Ordenar por Bioma (Heap Sort)\n");
245         printf("8 - Ordenar por Município (Heap Sort)\n");
246         printf("9 - Ordenar por Data (Heap Sort)\n");
247         printf("0 - Sair\n");
248         printf("Escolha uma opção: ");
249         scanf("%d", &opcao);

```

O controle das opções foi implementado por meio de uma estrutura switch (Figura 14, linhas 251 a 303). Cada case executava o respectivo algoritmo de ordenação, Quick Sort, Bubble Sort ou Heap Sort, recebendo como parâmetros o ponteiro focos, o total de registro e o método de comparação correspondente (comparar_bioma, comparar_municipio ou comparar_data). Após a ordenação, o método exibir_focos() era chamado para apresentar os resultados conforme a formatação previamente definida. Como já descrito, o método de comparação determinava se a ordenação ocorreria em ordem alfabética ou crescente, de acordo com campo selecionado. O case 0 encerrava o programa, finalizando o loop e liberando recursos utilizados.

Figura 14 – Terceira parte do código main: responsável pela tomada de decisão e da apresentação de resultados no console.

```

251     switch(opcao) {
252         case 1:
253             quick_sort(focos, total, comparar_bioma);
254             printf("\n--- DADOS ORDENADOS POR BIOMA (Quick Sort) ---");
255             exhibir_focos(focos, total);
256             break;
257         case 2:
258             quick_sort(focos, total, comparar_municipio);
259             printf("\n--- DADOS ORDENADOS POR MUNICÍPIO (Quick Sort) ---");
260             exhibir_focos(focos, total);
261             break;
262         case 3:
263             quick_sort(focos, total, comparar_data);
264             printf("\n--- DADOS ORDENADOS POR DATA (Quick Sort) ---");
265             exhibir_focos(focos, total);
266             break;
267         case 4:
268             bubble_sort(focos, total, comparar_bioma);
269             printf("\n--- DADOS ORDENADOS POR BIOMA (Bubble Sort) ---");
270             exhibir_focos(focos, total);
271             break;
272         case 5:
273             bubble_sort(focos, total, comparar_municipio);
274             printf("\n--- DADOS ORDENADOS POR MUNICÍPIO (Bubble Sort) ---");
275             exhibir_focos(focos, total);
276             break;
277         case 6:
278             bubble_sort(focos, total, comparar_data);
279             printf("\n--- DADOS ORDENADOS POR DATA (Bubble Sort) ---");
280             exhibir_focos(focos, total);
281             break;
282         case 7:
283             heap_sort(focos, total, comparar_bioma);
284             printf("\n--- DADOS ORDENADOS POR BIOMA (Heap Sort) ---");
285             exhibir_focos(focos, total);
286             break;
287         case 8:
288             heap_sort(focos, total, comparar_municipio);
289             printf("\n--- DADOS ORDENADOS POR MUNICÍPIO (Heap Sort) ---");
290             exhibir_focos(focos, total);
291             break;
292         case 9:
293             heap_sort(focos, total, comparar_data);
294             printf("\n--- DADOS ORDENADOS POR DATA (Heap Sort) ---");
295             exhibir_focos(focos, total);
296             break;
297         case 0:
298             printf("Encerrando...\n");
299             break;
300         default:
301             printf("Opção inválida!\n");
302     }
303 } while(opcao != 0);
304

```

Por fim, o programa main encerrou-se com a liberação da memória alocada e o retorno do valor 0 (linhas 305 a 307), conforme mostrado na Figura 15, indicando a execução bem-sucedida do sistema.

Figura 15 – Finalização do código main.

```
305     free(focos);  
306     return 0;  
307 }
```

5.2 Testes do programa

Os testes realizados demonstraram o funcionamento adequado do programa, sendo que todos os resultados esperados foram exibidos no console.

5.2.1 Funcionamento do programa

Inicialmente, o nome de um dos arquivos foi propositalmente alterado com o objetivo de verificar se o programa exibiria a mensagem de erro correspondente. A Figura 16 apresentou corretamente a mensagem já mostrada na Figura 3, confirmando que o tratamento de erro relacionado à abertura de arquivos executou-se conforme o esperado.

Figura 16 – Impressão no console quando o arquivo não é encontrado.

```
Carregando dados...  
Erro ao abrir arquivo: focos_br_al_ref_2023.csv
```

Após restaurar os nomes corretos dos arquivos, o programa exibiu no console o menu de opções disponível ao usuário, conforme ilustrado na Figura 17. Esse menu, previamente apresentado na Figura 13, demonstrou que, quando os arquivos se encontravam nomeados e localizados corretamente, o sistema prosseguia normalmente para a fase de seleção das opções de ordenação.

Em seguida, foi realizado um teste para validar o encerramento do programa ao selecionar a opção correspondente ao dígito zero. Conforme evidenciado na Figura 17, a mensagem de encerramento apresentada e o programa foi finalizado com sucesso. Adicionalmente, testou-se o comportamento do sistema ao inserir uma opção inexistente; nesse caso, o console exibiu a mensagem “Opção inválida!” (Figura 18).

Observou-se ainda que, após o recebimento da opção inválida, o programa retornou automaticamente ao menu principal, em vez de ser finalizado.

Figura 17 – Impressão apresentado no console do menu de opções e a escolha da opção zero.

```

=== SISTEMA DE ORDENAÇÃO DE FOCOS ===
1 - Ordenar por Bioma (Quick Sort)
2 - Ordenar por Município (Quick Sort)
3 - Ordenar por Data (Quick Sort)
4 - Ordenar por Bioma (Bubble Sort)
5 - Ordenar por Município (Bubble Sort)
6 - Ordenar por Data (Bubble Sort)
7 - Ordenar por Bioma (Heap Sort)
8 - Ordenar por Município (Heap Sort)
9 - Ordenar por Data (Heap Sort)
0 - Sair
Escolha uma opção: 0
Encerrando...

```

Figura 18 – Resultado do console ao se digitar uma opção inválida.

```

Escolha uma opção: 14
Opção inválida!

```

5.2.2 Validações e iterações do Bubble Sort

Para iniciar os testes, foi selecionada a opção correspondente ao algoritmo Bubble Sort. Uma vez escolhida a opção desejada, o programa executou o processo de ordenação conforme o esperado.

Na primeira validação, optou-se pela ordenação alfabética pelo campo bioma. A Figura 19 apresenta as primeiras linhas do resultado, evidenciando que a ordenação foi realizada corretamente. Nessa execução, foram contabilizadas 130.816 iterações e 19.845 trocas. A Figura 20 mostra as últimas linhas da lista ordenada e confirma o total de registros processados.

Em seguida, foi selecionada a ordenação pelo campo município. Os dados foram exibidos em ordem alfabética, conforme previsto. Nessa execução, o programa registrou 130.816 iterações e 61.588 trocas, processando um total de 512 registros.

Por fim, realizou-se o teste de ordenação pela data. A listagem apresentou as ocorrências em ordem cronológica crescente, iniciando pelas queimadas mais antigas

e finalizando pelas mais recentes. Para essa operação, foram contabilizadas 130.816 iterações e 61.329 trocas.

Figura 19 – Ordenação por Bioma pelo método Bubble Sort ordenado por bioma.

Escolha uma opção: 4

Iterações realizadas: 130816
Trocias realizadas: 19845

--- DADOS ORDENADOS POR BIOMA (Bubble Sort) ---

ID	Data	Lat	Lon	Bioma	Município	Estado
1615355323	2023-01-05 16:05:00	-9.206570	-37.594181	Caatinga	CANAPI	ALAGOAS
1617653502	2023-01-26 16:11:00	-10.046540	-36.382252	Caatinga	CORURIBE	ALAGOAS
1617653626	2023-01-26 16:11:00	-10.045230	-36.373089	Caatinga	CORURIBE	ALAGOAS
1617867894	2023-01-28 15:58:00	-10.063330	-36.815971	Caatinga	OLHO D'ÁGUA GRANDE	ALAGOAS
1617867895	2023-01-28 15:58:00	-10.064380	-36.808868	Caatinga	OLHO D'ÁGUA GRANDE	ALAGOAS
1617867912	2023-01-28 15:58:00	-9.784970	-36.869751	Caatinga	GIRAU DO PONCIANO	ALAGOAS
1617867913	2023-01-28 15:58:00	-9.786960	-36.884430	Caatinga	GIRAU DO PONCIANO	ALAGOAS
1617868012	2023-01-28 15:58:00	-9.773910	-36.871281	Caatinga	GIRAU DO PONCIANO	ALAGOAS
1618427461	2023-02-02 16:15:00	-9.791890	-36.761490	Caatinga	GIRAU DO PONCIANO	ALAGOAS
1618720251	2023-02-04 16:02:00	-9.850680	-37.020210	Caatinga	TRAIPU	ALAGOAS
1621094171	2023-02-21 16:34:00	-9.491020	-36.643970	Caatinga	PALMEIRA DOS ÍNDIOS	ALAGOAS
1622159795	2023-03-02 16:24:00	-10.030980	-36.901421	Caatinga	TRAIPU	ALAGOAS

Assim, verificou-se que o algoritmo Bubble Sort foi executado corretamente em todos os cenários de teste, realizando a ordenação conforme o critério selecionado e apresentando o número de iterações e trocas ao final do processamento.

Figura 20 – Últimas linhas da ordenação por Bubble Sort ordenado por bioma.

1678201382	2024-03-12 16:09:00	-9.916450	-37.078701	Mata Atlântica	TRAIPU	ALAGOAS
1647730128	2023-08-31 16:13:00	-9.087280	-36.085720	Mata Atlântica	UNIÃO DOS PALMARES	ALAGOAS
1679502997	2024-03-20 16:43:00	-9.102910	-35.975529	Mata Atlântica	UNIÃO DOS PALMARES	ALAGOAS
1732256715	2024-11-10 16:52:00	-9.193940	-36.144459	Mata Atlântica	UNIÃO DOS PALMARES	ALAGOAS
1736464383	2024-12-12 16:47:00	-9.080270	-36.103291	Mata Atlântica	UNIÃO DOS PALMARES	ALAGOAS
1736464448	2024-12-12 16:47:00	-9.190820	-36.102638	Mata Atlântica	UNIÃO DOS PALMARES	ALAGOAS
1736464739	2024-12-12 16:47:00	-9.189400	-36.092640	Mata Atlântica	UNIÃO DOS PALMARES	ALAGOAS
1737230152	2024-12-17 16:47:00	-9.127310	-36.043209	Mata Atlântica	UNIÃO DOS PALMARES	ALAGOAS
1737230593	2024-12-17 16:47:00	-9.136470	-36.041859	Mata Atlântica	UNIÃO DOS PALMARES	ALAGOAS
1679691432	2024-03-22 16:26:00	-9.374010	-36.199291	Mata Atlântica	VIÇOSA	ALAGOAS

Total de registros: 512

5.2.3 Validação e iteração do Quick Sort

A Figura 21 apresentou o processo de ordenação utilizando o método Quick Sort, aplicando inicialmente ao campo município. Observou-se que os municípios foram organizados corretamente em ordem alfabética., evidenciando o funcionamento adequado do algoritmo. Nesse cenário, o número de iterações foi de 7.500 e o número de trocas realizadas foi de 2.268.

De maneira semelhante, ao selecionar a ordenação pelo campo bioma, os dados foram exibidos em ordem alfabética conforme esperado. Para este caso, o algoritmo realizou 84.835 iterações e 627 trocas.

Por fim, ao ordenar pelo campo data, as queimadas mais antigas foram exibidas no início da lista, enquanto as queimadas mais recentes apareceram ao final, confirmando o comportamento correto do método. Nessa execução, observaram-se 44.960 iterações e 44.332 trocas. Em todas as situações, o total de registros processados permaneceu constante, totalizando 512 registros.

Figura 21 – Método de ordenação por Quick Sort ordenado por município mostrando as iterações e trocas realizadas.

Escolha uma opção: 2

Iterações realizadas: 7500
Trocias realizadas: 2268

--- DADOS ORDENADOS POR MUNICÍPIO (Quick Sort) ---

ID	Data	Lat	Lon	Bioma	Município	Estado
1708837703	2024-08-12 17:09:00	-9.650300	-36.351479	Mata Atlântica	ANADIA	ALAGOAS
1670855561	2024-01-30 16:45:00	-9.687330	-36.659191	Caatinga	ARAPIRACA	ALAGOAS
1736224856	2024-12-10 17:04:00	-9.450930	-35.976841	Mata Atlântica	ATALAIA	ALAGOAS
1662378804	2023-11-26 16:07:00	-9.546340	-36.088120	Mata Atlântica	ATALAIA	ALAGOAS
1662378805	2023-11-26 16:07:00	-9.544250	-36.072029	Mata Atlântica	ATALAIA	ALAGOAS
1662777470	2023-12-01 16:17:00	-9.530730	-36.171532	Mata Atlântica	ATALAIA	ALAGOAS
1662777471	2023-12-01 16:17:00	-9.529260	-36.160690	Mata Atlântica	ATALAIA	ALAGOAS
1662777472	2023-12-01 16:17:00	-9.527970	-36.166451	Mata Atlântica	ATALAIA	ALAGOAS
1665077482	2023-12-23 16:45:00	-9.427440	-35.995449	Mata Atlântica	ATALAIA	ALAGOAS
1666102067	2024-01-01 16:24:00	-9.492370	-35.998890	Mata Atlântica	ATALAIA	ALAGOAS
1670855562	2024-01-30 16:45:00	-9.383110	-35.948399	Mata Atlântica	ATALAIA	ALAGOAS
1739313316	2024-12-30 17:11:00	-9.529280	-36.038212	Mata Atlântica	ATALAIA	ALAGOAS
1732948746	2024-11-15 16:54:00	-9.469810	-36.204540	Mata Atlântica	ATALAIA	ALAGOAS
1732948777	2024-11-15 16:54:00	-9.471110	-36.213459	Mata Atlântica	ATALAIA	ALAGOAS
1733827012	2024-11-22 16:37:00	-9.535810	-35.985050	Mata Atlântica	ATALAIA	ALAGOAS
1733827246	2024-11-22 16:37:00	-9.534100	-35.972710	Mata Atlântica	ATALAIA	ALAGOAS
1734476142	2024-11-27 16:39:00	-9.468100	-35.957088	Mata Atlântica	ATALAIA	ALAGOAS
1650226152	2023-09-12 16:24:00	-9.349840	-35.505630	Mata Atlântica	BARRA DE SANTO ANTÔNIO	ALAGOAS
1729188377	2024-10-19 16:56:00	-9.413150	-35.519531	Mata Atlântica	BARRA DE SANTO ANTÔNIO	ALAGOAS
1718866624	2024-09-07 16:39:00	-9.358180	-35.524460	Mata Atlântica	BARRA DE SANTO ANTÔNIO	ALAGOAS
1650226153	2023-09-12 16:24:00	-9.340800	-35.507000	Mata Atlântica	BARRA DE SANTO ANTÔNIO	ALAGOAS
1675178493	2024-02-20 16:28:00	-9.401730	-35.557209	Mata Atlântica	BARRA DE SANTO ANTÔNIO	ALAGOAS
1675178494	2024-02-20 16:28:00	-9.369750	-35.525070	Mata Atlântica	BARRA DE SANTO ANTÔNIO	ALAGOAS
1739313099	2024-12-30 17:11:00	-9.621050	-37.090340	Caatinga	BATALHA	ALAGOAS
1715577860	2024-08-29 17:09:00	-9.741190	-37.001530	Caatinga	BATALHA	ALAGOAS
1678201388	2024-03-12 16:09:00	-9.752220	-37.110741	Caatinga	BATALHA	ALAGOAS
1678201389	2024-03-12 16:09:00	-9.749150	-37.087521	Caatinga	BATALHA	ALAGOAS

5.2.4 Validação e iteração por Heap Sort

A Figura 22. Apresentou o processo de ordenação utilizando o método Heap Sort, aplicado inicialmente ao campo data. Observou-se que, conforme esperado, as queimadas mais antigas foram posicionadas no início da lista, enquanto as mais

recentes apareceram ao final. Nessa execução, o número de iteração foi de 4.653 e o número de trocas realizadas foi de 4.397.

Da mesma forma, as ordenações pelos campos bioma e município também exibiram os registros em ordem alfabética, conforme previsto. Quando aplicada a ordenação por bioma, o algoritmo realizou 2.154 iterações e 1.898 trocas. Já para a ordenação por município, foram executadas 4.428 iterações e 4.172 trocas. Em todos os casos, o total de registros processados permaneceu constante, totalizando 512 entradas.

Figura 22 – Método de ordenação por Heap Sort ordenado por data e mostrando as iterações e trocas realizadas

Escolha uma opção: 9

Iterações realizadas: 4653
Trocas realizadas: 4397

--- DADOS ORDENADOS POR DATA (Heap Sort) ---

ID	Data	Lat	Lon	Bioma	Município	Estado
1615355323	2023-01-05 16:05:00	-9.206570	-37.594181	Caatinga	CANAPI	ALAGOAS
1615355326	2023-01-05 16:05:00	-9.024120	-35.301449	Mata Atlântica	MARAGOGI	ALAGOAS
1615729756	2023-01-08 16:34:00	-9.930060	-36.286800	Mata Atlântica	TEOTÔNIO VILELA	ALAGOAS
1615729922	2023-01-08 16:34:00	-9.918810	-36.288681	Mata Atlântica	TEOTÔNIO VILELA	ALAGOAS
1615832021	2023-01-09 15:39:00	-8.928570	-35.890110	Mata Atlântica	IBATEGUARA	ALAGOAS
1615832022	2023-01-09 15:39:00	-8.929110	-35.912289	Mata Atlântica	IBATEGUARA	ALAGOAS
1615832020	2023-01-09 15:39:00	-8.925630	-35.883350	Mata Atlântica	IBATEGUARA	ALAGOAS
1616984589	2023-01-19 16:09:00	-9.821370	-36.430031	Mata Atlântica	JUNQUEIRO	ALAGOAS
1617653626	2023-01-26 16:11:00	-10.045230	-36.373089	Caatinga	CORURIBE	ALAGOAS
1617653610	2023-01-26 16:11:00	-9.343680	-35.589111	Mata Atlântica	SÃO LUÍS DO QUITUNDE	ALAGOAS
1617653611	2023-01-26 16:11:00	-9.344980	-35.598141	Mata Atlântica	SÃO LUÍS DO QUITUNDE	ALAGOAS
1617653502	2023-01-26 16:11:00	-10.046540	-36.382252	Caatinga	CORURIBE	ALAGOAS
1617653612	2023-01-26 16:11:00	-9.615500	-35.816219	Mata Atlântica	SANTA LUZIA DO NORTE	ALAGOAS
1617653575	2023-01-26 16:11:00	-8.877090	-35.487019	Mata Atlântica	JACUÍPE	ALAGOAS
1617867913	2023-01-28 15:58:00	-9.786960	-36.884430	Caatinga	GIRAU DO PONCIANO	ALAGOAS
1617867912	2023-01-28 15:58:00	-9.784970	-36.869751	Caatinga	GIRAU DO PONCIANO	ALAGOAS
1617868012	2023-01-28 15:58:00	-9.773910	-36.871281	Caatinga	GIRAU DO PONCIANO	ALAGOAS
1617867894	2023-01-28 15:58:00	-10.063330	-36.815971	Caatinga	OLHO D'ÁGUA GRANDE	ALAGOAS
1617867895	2023-01-28 15:58:00	-10.064380	-36.808868	Caatinga	OLHO D'ÁGUA GRANDE	ALAGOAS
1618083055	2023-01-30 15:48:00	-8.935330	-35.294498	Mata Atlântica	MARAGOGI	ALAGOAS
1618083056	2023-01-30 15:48:00	-8.937850	-35.313202	Mata Atlântica	MARAGOGI	ALAGOAS
1618083057	2023-01-30 15:48:00	-8.939100	-35.306061	Mata Atlântica	MARAGOGI	ALAGOAS
1618427461	2023-02-02 16:15:00	-9.791890	-36.761490	Caatinga	GIRAU DO PONCIANO	ALAGOAS
1618720232	2023-02-04 16:02:00	-10.025780	-36.522461	Mata Atlântica	SÃO SEBASTIÃO	ALAGOAS
1618720251	2023-02-04 16:02:00	-9.850680	-37.020210	Caatinga	TRAIPU	ALAGOAS
1619406154	2023-02-09 16:17:00	-10.031740	-36.233860	Mata Atlântica	CORURIBE	ALAGOAS
1619974009	2023-02-13 15:52:00	-8.916160	-35.653809	Mata Atlântica	NOVO LINO	ALAGOAS
1620416872	2023-02-16 16:20:00	-9.117570	-37.740299	Caatinga	MATA GRANDE	ALAGOAS

6 DISCUSSÃO E CONCLUSÃO

Neste trabalho, realizou-se uma organização e análise dos dados de queimadas ocorridas no estado de Alagoas. A utilização de técnicas de ordenação mostrou-se essencial para estruturar essas informações de maneira a facilitar a interpretação dos resultados e a extração de padrões relevantes. Ao ordenar os registros, torna-se possível identificar com maior clareza em quais biomas as queimadas ocorreram com maior frequência, quais municípios apresentaram maior número de focos e quais períodos temporais concentraram mais eventos. Essas informações não apenas permitem compreender o comportamento histórico das queimadas, mas também fornecem subsídios para políticas públicas de prevenção e mitigação de impactos ambientais. A ordenação adequada de registros de queimadas não apenas facilita análises estatísticas, mas também é fundamental para o desenvolvimento de sistemas automatizados de detecção e resposta a desastres ambientais. Em contextos onde as informações são coletadas continuamente por satélites, a capacidade de organizar e processar dados em tempo hábil pode representar a diferença entre uma resposta preventiva e uma ação tardia.

O processo de ordenação de dados, muitas vezes considerado uma etapa simples em projetos de ciência de dados, desempenha um papel crucial quando se trabalha com grandes volumes de informações. No caso de bancos ambientais, que frequentemente reúnem milhões de registros coletados por satélites, sensores e bases meteorológicas, a escolha do algoritmo de ordenação pode afetar diretamente o tempo de processamento e a eficiência geral do sistema. Assim, a análise comparativa entre diferentes métodos de ordenação – como Bubble Sort, Quick Sort e Heap Sort – torna-se uma oportunidade valiosa para compreender não apenas o desempenho teórico desses algoritmos, mas também suas implicações práticas no tratamento de dados reais.

No contexto específico deste estudo, o número de registros analisados correspondeu às queimadas registradas em Alagoas ao longo de dois anos (2023 e 2024). Essa quantidade, embora relativamente pequena, permite observar com clareza as diferenças de comportamento entre os algoritmos, sem exigir alta capacidade computacional. Contudo, é importante ressaltar que, em um cenário mais amplo – por exemplo, abrangendo todo o território nacional ou períodos históricos

mais extensos –, a eficiência do algoritmo torna-se um fator determinante. Em bases com centenas de milhares de registros, a diferença entre um algoritmo de complexidade quadrática ($O(n^2)$) e outro de complexidade logarítmica ($O(n \log n)$) pode representar horas de processamento e consumo excessivo de recursos.

Além do volume de dados, outro aspecto relevante é a natureza das variáveis ordenadas. Em bases ambientais, campos como “bioma”, “município” e “data” possuem características distintas que podem impactar o comportamento dos algoritmos. O campo “bioma”, por exemplo, tende a apresentar um número limitado de categorias (Caatinga, Mata Atlântica, Cerrado etc.), o que favorece uma distribuição mais concentrada e reduz o número de trocas em certos algoritmos. Já o campo “município”, composto por centenas de valores distintos, exige mais comparações e resulta em maior custo computacional, especialmente em métodos menos eficientes. A ordenação por “data”, por sua vez, reflete um comportamento quase sequencial, podendo beneficiar algoritmos que exploram relações de proximidade entre elementos adjacentes.

Essa análise detalhada reforça que o desempenho de um algoritmo não depende apenas de sua complexidade teórica, mas também da estrutura interna dos dados e da forma como são distribuídos. Por isso, compreender o contexto e o tipo de variável a ser ordenada é fundamental na escolha do método mais adequado. Em sistemas reais de monitoramento ambiental, a aplicação de algoritmos mais eficientes pode viabilizar a atualização quase em tempo real dos dados, facilitando o acompanhamento de novos focos de incêndio e permitindo a tomada de decisões rápidas por órgãos de controle.

Em um cenário prático, a diferença entre os algoritmos testados pode ser observada na quantidade de iterações e trocas necessárias para alcançar a ordenação completa dos registros. O Bubble Sort, por exemplo, ainda que de fácil implementação, realiza comparações exaustivas entre elementos adjacentes, o que o torna inviável para grandes conjuntos de dados. O Quick Sort, por outro lado, apresenta um desempenho superior por utilizar a divisão recursiva dos elementos em subgrupos, mas seu comportamento depende da escolha do pivô e da distribuição inicial dos dados. Já o Heap Sort destaca-se por manter desempenho consistente, independente da ordem inicial, o que o torna especialmente indicado em situações em que a previsibilidade de tempo de execução é desejada.

Em termos de aplicabilidade, é possível imaginar um cenário em que o volume de registros ambientais cresça exponencialmente devido à expansão das tecnologias de sensoriamento remoto e à integração de bancos de dados nacionais. Nessa situação, o uso de algoritmos mais simples, como o Bubble Sort, deixaria de ser viável, pois o tempo necessário para processar todos os registros seria desproporcional. Por outro lado, métodos mais sofisticados, como o Heap Sort ou o Quick Sort, garantiriam tempos de resposta mais curtos e maior eficiência, permitindo que análises complexas fossem executadas em intervalos compatíveis com as necessidades operacionais de monitoramento ambiental.

Portanto, a escolha do algoritmo de ordenação não deve ser vista apenas como uma questão técnica, mas também como uma decisão estratégica dentro do processo de análise de dados. Em ambientes científicos e de gestão ambiental, onde a precisão e a rapidez das informações são fundamentais, compreender a eficiência e o comportamento de diferentes algoritmos é uma competência essencial. A partir dessa perspectiva, o presente estudo contribui não apenas para avaliar a eficiência dos métodos de ordenação em uma base de dados específica, mas também para destacar a importância da computação eficiente como ferramenta de apoio à sustentabilidade e à tomada de decisões ambientais.

6.1 Resumo dos dados de iteração e trocas

Tabela 1 – Número de iterações e trocas para cada algoritmo com 512 registros analisados.

Método	Critério	Iterações	Trocas
Bubble Sort	Bioma	130.816	19.845
	Município	130.816	61.588
	Data	130.816	51.329
Quick Sort	Bioma	84.835	627
	Município	7.500	2.268
	Data	44.960	44.332
Heap Sort	Bioma	2.154	1.898
	Município	4.428	4.172
	Data	4.653	4.397

Como todos os métodos trabalharam sobre a mesma base de dados, o número de registros de entrada foi idêntico em todas as ordenações. Dessa forma, as diferenças observadas no número de iterações e de trocas resultaram exclusivamente da complexidade e do comportamento de cada algoritmo. A Tabela 1 apresenta um resumo dos valores de iteração e de troca obtidos para os 512 registros analisados.

6.2 Bubble Sort

Como já mencionado anteriormente, o Bubble Sort realiza a ordenação comparando pares adjacentes e deslocando os maiores valores para o final da lista a cada iteração, garantindo que, ao fim de cada ciclo externo, o maior elemento restante esteja corretamente posicionado. No caso da base analisada, composta por 512 registros de queimadas no estado de Alagoas, o algoritmo apresentou custos computacionais elevados.

Quando os dados foram ordenados alfabeticamente por bioma, o programa realizou 130.816 iterações e 19.845 trocas. Ao ordenar por município, observaram-se 130.816 iterações e 61.588 trocas, e ao ordenar por data, registraram-se 130.816 iterações e 61.396 trocas.

Foi possível observar que o número de iterações permaneceu constante, independentemente do critério de ordenação, enquanto o número de trocas variou conforme a distribuição inicial dos dados. Esse comportamento era esperado, pois o Bubble Sort possui complexidade quadrática $O(n^2)$ e sempre realiza o mesmo número de comparações para um conjunto de dados de tamanho fixo. Já o número de trocas depende diretamente da quantidade de elementos fora de ordem no arranjo original.

A quantidade de iterações (130.816) pode ser calculada analiticamente pela expressão:

$$iterações = \frac{n(n-1)}{2}$$

O elevado número de operações evidencia a ineficiência inerente ao algoritmo para conjuntos de dados maiores, mesmo em um cenário relativamente pequeno como o analisado, reforçando sua limitação prática em aplicações reais.

6.3 Quick Sort

O Quick Sort ordena os elementos dividindo recursivamente a lista em sublistas menores a partir de um pivô, separando valores menores e maiores que esse elemento e repetindo o processo até que toda a lista esteja organizada. Devido à sua complexidade média $O(n \log n)$, espera-se que apresente um desempenho significativamente melhor em relação ao Bubble Sort.

Ordenando por bioma, o algoritmo realizou 84.442 iterações e 628 trocas. Na ordenação por município, registraram-se 7.520 iterações e 2.260 trocas, enquanto na ordenação por data foram observadas 30.171 iterações e 27.930 trocas.

Diferentemente do Bubble Sort, o Quick Sort não apresentou o mesmo número de iterações em todos os testes. Essa variação ocorreu devido à escolha dos pivôs e à distribuição inicial dos dados, fatores que influenciam diretamente o número de subdivisões realizadas ao longo da execução. Quando o pivô divide a lista em partes aproximadamente iguais, o algoritmo tende a executar um número reduzido de operações. Entretanto, quando a divisão ocorre de forma assimétrica, o número de chamadas recursivas aumenta, elevando o número total de operações.

Ainda assim, mesmo em seus cenários menos favoráveis, o Quick Sort demonstrou desempenho superior ao Bubble Sort, confirmando sua eficiência teórica.

6.4 Heap Sort

O Heap Sort organiza os dados por meio da construção de uma estrutura de heap, permitindo que o maior (ou menor) elemento seja sistematicamente removido e reposicionado até completar a ordenação. Esse algoritmo apresenta complexidade $O(n \log n)$ de forma consistente, independentemente da distribuição inicial dos dados.

Ao ordenar os registros por bioma, observou-se 2.154 iterações e 1.898 trocas. Na ordenação por município, ocorreram 4.428 iterações e 4.172 trocas, enquanto na ordenação por data registraram-se 4.653 iterações e 4.397 trocas.

Percebe-se que a variação tanto de iterações quanto de trocas foi pequena entre os diferentes tipos de ordenação, indicando maior estabilidade do algoritmo em relação ao Quick Sort. Essa estabilidade decorre da estrutura do heap, que garante operações balanceadas e não depende de pivôs ou da ordem inicial dos dados.

6.5 Conclusão

De maneira geral, os resultados confirmaram o comportamento esperado para cada algoritmo. O Bubble Sort apresentou o maior número de operações, indicando desempenho significativamente inferior e baixa escalabilidade. O Quick Sort demonstrou maior eficiência, embora com variação em função da estrutura inicial dos dados e da seleção dos pivôs. Já o Heap Sort exibiu o comportamento mais estável, com baixo número de iterações e trocas em todos os cenários testados. Dessa forma, para aplicações envolvendo grandes volumes de dados ambientais, como registros históricos de queimadas, algoritmos $O(n \log n)$ — especialmente o Heap Sort — mostram-se mais adequados por garantir maior desempenho e menor dependência da configuração inicial dos dados.

Considerando os resultados obtidos, é possível propor algumas perspectivas de continuidade para este estudo. Uma ampliação natural seria a aplicação dos mesmos algoritmos em conjuntos de dados mais extensos, abrangendo todo o território nacional ou períodos superiores a cinco anos, o que permitiria avaliar o comportamento dos métodos em situações de alta escala. Além disso, a comparação com outros algoritmos de complexidade $O(n \log n)$, como o Merge Sort ou o Timsort (utilizado em linguagens de programação modernas), poderia enriquecer a análise ao incorporar fatores como estabilidade e otimização interna. Também seria interessante medir o tempo real de processamento em diferentes plataformas computacionais, a fim de verificar o impacto da implementação e do hardware no desempenho global.

7 RELATÓRIO COM AS LINHAS DE CÓDIGO DO PROGRAMA

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <windows.h>

#define MAX_REGISTROS 1000
#define MAX_STRING 100

typedef struct {
    int id_bdq;
    char foco_id[MAX_STRING];
    float lat;
    float lon;
    char data_pas[MAX_STRING];
    char pais[MAX_STRING];
    char estado[MAX_STRING];
    char municipio[MAX_STRING];
    char bioma[MAX_STRING];
} Foco;

// ===== COMPARADORES =====

int comparar_bioma(const void *a, const void *b) {
    return strcmp(((Foco *)a)->bioma, ((Foco *)b)->bioma);
}

int comparar_municipio(const void *a, const void *b) {
    return strcmp(((Foco *)a)->municipio, ((Foco *)b)->municipio);
}

int comparar_data(const void *a, const void *b) {
    return strcmp(((Foco *)a)->data_pas, ((Foco *)b)->data_pas);
}
```

```
}
```

```
// ===== BUBBLE SORT =====
```

```
void bubble_sort(Foco *focos, int total, int (*comparar)(const void *, const void *)) {
```

```
    long iteracoes = 0, trocas = 0;
```

```
    for (int i = 0; i < total - 1; i++) {
```

```
        for (int j = 0; j < total - i - 1; j++) {
```

```
            iteracoes++;
```

```
            if (comparar(&focos[j], &focos[j + 1]) > 0) {
```

```
                Foco temp = focos[j];
```

```
                focos[j] = focos[j + 1];
```

```
                focos[j + 1] = temp;
```

```
                trocas++;
```

```
            }
```

```
        }
```

```
    }
```

```
    printf("\nIterações realizadas: %ld\n", iteracoes);
```

```
    printf("Trocas realizadas: %ld\n", trocas);
```

```
}
```

```
// ===== QUICK SORT (com contagem)
```

```
=====
```

```
int particionar(Foco *focos, int baixo, int alto,
```

```
    int (*comparar)(const void *, const void *),
```

```
    long *iteracoes, long *trocas) {
```

```
    Foco pivo = focos[alto];
```

```
    int i = baixo - 1;
```

```
    for (int j = baixo; j < alto; j++) {
```

```
        (*iteracoes)++;
```

```
        if (comparar(&focos[j], &pivo) < 0) {
```

```

        i++;
        Foco temp = focos[i];
        focos[i] = focos[j];
        focos[j] = temp;
        (*trocas)++;
    }
}

Foco temp = focos[i + 1];
focos[i + 1] = focos[alto];
focos[alto] = temp;
(*trocas)++;

return i + 1;
}

void quick_sort_recursivo(Foco *focos, int baixo, int alto,
                        int (*comparar)(const void *, const void *),
                        long *iteracoes, long *trocas) {
    if (baixo < alto) {
        int pi = particionar(focos, baixo, alto, comparar, iteracoes, trocas);
        quick_sort_recursivo(focos, baixo, pi - 1, comparar, iteracoes, trocas);
        quick_sort_recursivo(focos, pi + 1, alto, comparar, iteracoes, trocas);
    }
}

void quick_sort(Foco *focos, int total, int (*comparar)(const void *, const void *)) {
    long iteracoes = 0, trocas = 0;
    quick_sort_recursivo(focos, 0, total - 1, comparar, &iteracoes, &trocas);
    printf("\nIterações realizadas: %ld\n", iteracoes);
    printf("Trocas realizadas: %ld\n", trocas);
}

// ===== HEAP SORT =====

```

```

void heapify(Foco *focos, int n, int i,
            int (*comparar)(const void *, const void *),
            long *iteracoes, long *trocas) {

    int maior = i;    // Inicializa o maior como raiz
    int esquerda = 2 * i + 1; // filho esquerdo
    int direita = 2 * i + 2;  // filho direito

    (*iteracoes)++;

    // Se o filho esquerdo é maior que a raiz
    if (esquerda < n && comparar(&focos[esquerda], &focos[maior]) > 0) {
        maior = esquerda;
    }

    // Se o filho direito é maior que o maior até agora
    if (direita < n && comparar(&focos[direita], &focos[maior]) > 0) {
        maior = direita;
    }

    // Se o maior não é a raiz
    if (maior != i) {
        // Troca a raiz com o maior elemento
        Foco temp = focos[i];
        focos[i] = focos[maior];
        focos[maior] = temp;
        (*trocas)++;

        // Recursivamente faz heapify na sub-árvore afetada
        heapify(focos, n, maior, comparar, iteracoes, trocas);
    }
}

void heap_sort(Foco *focos, int total, int (*comparar)(const void *, const void *)) {

```

```

long iteracoes = 0, trocas = 0;

// Constrói o heap (reorganiza o array)
for (int i = total / 2 - 1; i >= 0; i--) {
    heapify(focos, total, i, comparar, &iteracoes, &trocas);
}

// Extrai um elemento por vez do heap
for (int i = total - 1; i > 0; i--) {
    // Move a raiz atual para o final
    Foco temp = focos[0];
    focos[0] = focos[i];
    focos[i] = temp;
    trocas++;

    // Chama heapify no heap reduzido
    heapify(focos, i, 0, comparar, &iteracoes, &trocas);
}

printf("\nIterações realizadas: %ld\n", iteracoes);
printf("Trocas realizadas: %ld\n", trocas);
}

// ===== CSV =====
int carregar_csv(const char *filename, Foco *focos, int *total) {
    FILE *arquivo = fopen(filename, "r");
    if (!arquivo) {
        printf("Erro ao abrir arquivo: %s\n", filename);
        return 0;
    }
    char linha[500];
    fgets(linha, sizeof(linha), arquivo); // Pula cabeçalho

    int count = *total;

```

```

while (fgets(linha, sizeof(linha), arquivo) && count < MAX_REGISTROS) {
    linha[strcspn(linha, "\n")] = 0;

    char *token = strtok(linha, ",");
    if (token) focos[count].id_bdq = atoi(token);
    token = strtok(NULL, ",");
    if (token) strncpy(focos[count].foco_id, token, MAX_STRING - 1);
    token = strtok(NULL, ",");
    if (token) focos[count].lat = atof(token);
    token = strtok(NULL, ",");
    if (token) focos[count].lon = atof(token);
    token = strtok(NULL, ",");
    if (token) strncpy(focos[count].data_pas, token, MAX_STRING - 1);
    token = strtok(NULL, ",");
    if (token) strncpy(focos[count].pais, token, MAX_STRING - 1);
    token = strtok(NULL, ",");
    if (token) strncpy(focos[count].estado, token, MAX_STRING - 1);
    token = strtok(NULL, ",");
    if (token) strncpy(focos[count].municipio, token, MAX_STRING - 1);
    token = strtok(NULL, ",");
    if (token) strncpy(focos[count].bioma, token, MAX_STRING - 1);
    count++;
}
fclose(arquivo);
*total = count;
return 1;
}

// ===== EXIBIÇÃO =====

void exibir_focos(Foco *focos, int total) {
    printf("\n%-10s %-20s %-12s %-12s %-15s %-30s %-20s\n",
           "ID", "Data", "Lat", "Lon", "Bioma", "Município", "Estado");
    printf("-----\n");
}

```

```

for (int i = 0; i < total; i++) {
    printf("%-10d %-20s %-12.6f %-12.6f %-15s %-30s %-20s\n",
        focos[i].id_bdq,
        focos[i].data_pas,
        focos[i].lat,
        focos[i].lon,
        focos[i].bioma,
        focos[i].municipio,
        focos[i].estado);
}
printf("\nTotal de registros: %d\n", total);
}

// ===== MAIN =====
int main() {
    SetConsoleOutputCP(65001);
    Foco *focos = malloc(MAX_REGISTROS * sizeof(Foco));
    if (!focos) {
        printf("Erro ao alocar memória!\n");
        return 1;
    }

    int total = 0;

    printf("Carregando dados...\n");
    if (!carregar_csv("focos_br_al_ref_2023.csv", focos, &total)) {
        free(focos);
        return 1;
    }
    if (!carregar_csv("focos_br_al_ref_2024.csv", focos, &total)) {
        free(focos);
        return 1;
    }
}

```



```

int opcao;
do {
    printf("\n=== SISTEMA DE ORDENAÇÃO DE FOCOS ===\n");
    printf("1 - Ordenar por Bioma (Quick Sort)\n");
    printf("2 - Ordenar por Município (Quick Sort)\n");
    printf("3 - Ordenar por Data (Quick Sort)\n");
    printf("4 - Ordenar por Bioma (Bubble Sort)\n");
    printf("5 - Ordenar por Município (Bubble Sort)\n");
    printf("6 - Ordenar por Data (Bubble Sort)\n");
    printf("7 - Ordenar por Bioma (Heap Sort)\n");
    printf("8 - Ordenar por Município (Heap Sort)\n");
    printf("9 - Ordenar por Data (Heap Sort)\n");
    printf("0 - Sair\n");
    printf("Escolha uma opção: ");
    scanf("%d", &opcao);

    switch(opcao) {
        case 1:
            quick_sort(focos, total, comparar_bioma);
            printf("\n--- DADOS ORDENADOS POR BIOMA (Quick Sort) ---");
            exibir_focos(focos, total);
            break;
        case 2:
            quick_sort(focos, total, comparar_municipio);
            printf("\n--- DADOS ORDENADOS POR MUNICÍPIO (Quick Sort) ---");
            exibir_focos(focos, total);
            break;
        case 3:
            quick_sort(focos, total, comparar_data);
            printf("\n--- DADOS ORDENADOS POR DATA (Quick Sort) ---");
            exibir_focos(focos, total);
            break;
        case 4:

```

```
bubble_sort(focos, total, comparar_bioma);
printf("\n--- DADOS ORDENADOS POR BIOMA (Bubble Sort) ---");
exibir_focos(focos, total);
break;
case 5:
    bubble_sort(focos, total, comparar_municipio);
    printf("\n--- DADOS ORDENADOS POR MUNICÍPIO (Bubble Sort) ---");
    exibir_focos(focos, total);
    break;
case 6:
    bubble_sort(focos, total, comparar_data);
    printf("\n--- DADOS ORDENADOS POR DATA (Bubble Sort) ---");
    exibir_focos(focos, total);
    break;
case 7:
    heap_sort(focos, total, comparar_bioma);
    printf("\n--- DADOS ORDENADOS POR BIOMA (Heap Sort) ---");
    exibir_focos(focos, total);
    break;
case 8:
    heap_sort(focos, total, comparar_municipio);
    printf("\n--- DADOS ORDENADOS POR MUNICÍPIO (Heap Sort) ---");
    exibir_focos(focos, total);
    break;
case 9:
    heap_sort(focos, total, comparar_data);
    printf("\n--- DADOS ORDENADOS POR DATA (Heap Sort) ---");
    exibir_focos(focos, total);
    break;
case 0:
    printf("Encerrando...\n");
    break;
default:
    printf("Opção inválida!\n");
```

```
    }  
} while(opcao != 0);  
  
free(focos);  
return 0;  
}
```

8 BIBLIOGRAFIA

CORMEN, T. H. et al. Algoritmos: Teoria e Prática. 3. ed. Rio de Janeiro: Elsevier, 2012.

INPE – Instituto Nacional de Pesquisas Espaciais. Banco de Dados de Queimadas – Estados do Brasil (2023–2024). Disponível em: <https://dataserver-coids.inpe.br/queimadas/>. Acesso em: 10 nov. 2025.

KERNIGHAN, B. W.; RITCHIE, D. M. A Linguagem de Programação C. 2. ed. São Paulo: Pearson, 2014.

KNUTH, D. E. The Art of Computer Programming: Sorting and Searching. 2. ed. Addison-Wesley, 1998.

MANZANO, J. A. N. Estruturas de Dados e Algoritmos. São Paulo: Érica, 2017.

PRESSMAN, R. S. Engenharia de Software: Uma Abordagem Profissional. 8. ed. Porto Alegre: AMGH, 2016.

SEDGEWICK, R.; WAYNE, K. Algorithms. 4. ed. Boston: Addison-Wesley, 2011.

SOMMERVILLE, I. Engenharia de Software. 9. ed. São Paulo: Pearson, 2011.

TANENBAUM, A. S. Structured Computer Organization. 6. ed. New Jersey: Pearson, 2015.

TENENBAUM, A. M.; LANGSAM, Y.; AUGENSTEIN, M. J. Estruturas de Dados Usando C. São Paulo: Pearson, 2009.

WIRTH, N. Algoritmos + Estruturas de Dados = Programas. Rio de Janeiro: LTC, 1986.



FICHA DAS ATIVIDADES PRÁTICAS SUPERVISIONADAS - APS

NOME: Mariana Kobori Gabrielli

TURMA: CC3A68

RA: R192512

CURSO: Ciência da Computação

CAMPUS: Paraíso

SEMESTRE: 3

TURN O: Manhã

CÓDIGO DA ATIVIDADE:

SEMESTRE: 3

ANO GRADE: 2025

[illegible]

(1) Horas atribuídas de acordo com o regulamento das Atividades Práticas Supervisionadas do curso.

TOTAL DE HORAS ATRIBUÍDAS:	21 horas
----------------------------	----------

AVALIAÇÃO:

Aprovado ou Reprovado

NOTA:

DATA:

CARIMBO E ASSINATURA DO COORDENADOR DO CURSO



FICHA DAS ATIVIDADES PRÁTICAS SUPERVISIONADAS - APS

NOME: Gabriel H P Andrade

TURMA: CC4

RA: G73GHC3

CURSO: Ciência da computação

CAMPUS: Vergueiro

SEMESTRE:

TURN O: Manhã

CÓDIGO DA ATIVIDADE: 77B1

SEMESTRE: 4 Semestre

ANO GRADE: 2025

[illegible]

(1) Horas atribuídas de acordo com o regulamento das Atividades Práticas Supervisionadas do curso.

TOTAL DE HORAS ATRIBUÍDAS:

AVALIAÇÃO:

Aprovado ou Reprovado

NOTA:

DATA:

CARIMBO E ASSINATURA DO COORDENADOR DO CURSO